High Bandwidth Flash for Large Language Model Inference: Prefill–Decode Mechanics, System Integration, and the Memory-Hierarchy Architecture Debate

M. Nisha Angeline*

Professor and Head, Department of Electronics and Communication Engineering,
Velalar College of Engineering and Technology, Thindal, Erode, Tamil Nadu, India
*Corresponding author: nishavlsidesign@gmail.com

Abstract

The memory system, not arithmetic, now bounds the cost and latency of large language model (LLM) inference. High Bandwidth Memory (HBM) supplies terabytes per second but only tens of gigabytes per stack, forcing trillion-parameter mixture-of-experts (MoE) models to be sharded across many accelerator packages purely for capacity. High Bandwidth Flash (HBF) — vertically stacked 3D NAND dies joined by through-silicon vias to a logic base die and exposed through an HBM-like wide interface — has been proposed to deliver roughly an order of magnitude more capacity per stack at HBM-class read bandwidth, at the price of microsecond-scale access latency and finite write endurance. This paper develops a first-principles analysis of where HBF helps and where it hurts. We (i) derive the operational-intensity, capacity, and concurrency equations that govern the prefill and decode phases on a tiered HBM–HBF package, including the expert-touch probability that makes MoE decode traffic batch-dependent; (ii) propose a reference integration architecture covering the physical package, a read-optimised flash controller on the base die, a data-placement policy and the serving-runtime changes HBF requires; (iii) evaluate four package configurations on dense 70B and 1T-parameter MoE workloads with an open, parameterised analytical model under a 50 ms time-per-output-token service-level objective; and (iv) structure the ongoing architecture debate among HBM scale-out, on-package flash, CXL-attached pools, processing-near-memory and SRAM-centric dataflow designs. The model shows that HBF never improves per-package throughput for models that already fit in HBM, but for a 1T MoE at 32k context it enables single-package serving that HBM cannot provide and yields 1.75× (0.8 TB/s per HBF stack) to 4.2× (1.6 TB/s) higher per-package throughput at a four-package footprint. Endurance analysis shows that HBF is viable for weights and write-once prefix caches (≥8 years at one full rewrite per day) but not for the mutable KV cache (≈9 days at 1 GB/s on TLC). Sustaining HBM-class bandwidth at 20 µs array latency requires ≈1,000 concurrent 16 KiB reads per stack. We further design, exhaustively verify and synthesise (SkyWater 130 nm and ASAP7) array, Wallace, Dadda and radix-4 Booth multipliers of 4–16 bits, an exact-accumulating FP8 MAC and a systolic MAC array for the HBF base die, and compare them on area, delay, energy, leakage, ADP, EDP, GOPS/mm² and TOPS/W with and without HBF. An 8-bit array multiplier costs 17.9 fJ per operation in ASAP7; near-flash MAC engines cut weight-GEMV energy per token (MoE-1T, batch 1) by 23% relative to HBF with far compute and by 49% relative to a conventional HBM system, but a 5 W, 20 mm² base die sustains a decode batch of only ≈1.3 in 130 nm against ≈97 in 7 nm, making the base-die process — not the multiplier topology — the decisive design choice; with 4× internal bandwidth, near-flash engines raise four-package MoE-1T throughput from 219 to 659 tokens/s per package. We conclude that the most defensible near-term role for HBF is as a weight tier for capacity-constrained MoE deployments and for compute-bound prefill nodes in disaggregated serving, while HBM remains the home of the KV cache.

Keywords: High Bandwidth Flash (HBF); High Bandwidth Memory (HBM); 3D NAND; large language model inference; prefill and decode; KV cache; mixture of experts; memory hierarchy;

heterogeneous integration; through-silicon via; disaggregated serving; roofline model; array multiplier; Booth multiplier; Dadda tree; near-memory computing; open-source EDA.

1. Introduction

Transformer-based large language models [1], [2] have moved from research artefacts to infrastructure that serves billions of requests per day. Unlike training, which can be organised to reuse each weight across thousands of tokens, autoregressive inference generates one token per sequence per step. In this *decode* phase every generated token requires streaming the active model weights and the per-sequence key–value (KV) cache from memory, so the achievable throughput is set by memory bandwidth and the achievable batch by memory capacity [3], [4]. Gholami *et al.* [3] documented that peak accelerator FLOP/s has grown far faster than DRAM bandwidth and interconnect bandwidth over two decades; the resulting "memory wall" is now the dominant constraint for serving.

High Bandwidth Memory (HBM) [23], [24] has been the industry's answer: stacks of DRAM dies connected by through-silicon vias (TSVs) and placed beside the accelerator on a silicon interposer provide on the order of 1 TB/s per stack. Yet HBM capacity grows slowly — a 12-high HBM3E stack holds 36 GB — and its cost per bit is several times that of commodity DRAM. Meanwhile frontier models have grown to hundreds of billions or trillions of parameters, increasingly as sparse mixture-of-experts (MoE) networks [15]–[18], [44] whose *active* parameter count per token is small but whose *total* parameter count must still be resident. A 1T-parameter MoE stored in 8-bit precision occupies ≈1 TB before any KV cache is allocated, which exceeds the 288 GB of an eight-stack HBM3E package by more than 3×. Deployments therefore shard such models across four to sixteen packages for *capacity* rather than for compute, paying for idle FLOP/s, interconnect and power.

High Bandwidth Flash (HBF) attacks this capacity gap directly. Publicly announced in 2025 [29] and subsequently the subject of a standardisation memorandum between a NAND and an HBM vendor [30], HBF stacks 3D NAND core dies [26], [28] with TSVs onto a logic base die and exposes them to the host through an HBM-like wide, parallel interface. The stated goal is HBM-class *read* bandwidth with 8–16× the capacity per stack at comparable cost. NAND flash, however, differs from DRAM in three fundamental ways: (a) random read latency is tens of microseconds rather than tens of nanoseconds; (b) writes are page-granular, require prior block erasure and are orders of magnitude slower than reads; and (c) each cell tolerates only a finite number of program/erase (P/E) cycles [27]. Whether HBF is a revolution for inference or a niche capacity tier therefore depends on a careful accounting of *which* data structures are read-mostly, *which* phase of inference is bandwidth- versus compute-bound, and *how* latency can be hidden.

This paper provides that accounting. Its contributions are:

(1) **Phase-resolved analysis.** We derive closed-form expressions for the operational intensity of prefill and decode on a tiered HBM–HBF package, the minimum prefill chunk that keeps flash-resident weights compute-bound, the expert-touch probability that makes MoE weight traffic grow with batch size, and the Little's-law concurrency that a flash stack must sustain to reach its rated bandwidth (Section 3).

(2) **Reference integration architecture.** We specify a physical, controller-level and software-level integration of HBF, including a data-placement policy for weights, KV cache, prefix caches, adapters and activations, and quantify the endurance budget that rules each placement in or out (Section 4).

(3) **Quantitative evaluation.** Using an open, parameterised analytical model, we compare HBM-only and three HBM/HBF mixes for dense 70B and 1T-parameter MoE models across package

counts, context lengths and HBF bandwidth projections under a 50 ms time-per-output-token (TPOT) service-level objective (Sections 5–6).

(4) **Structured architecture debate.** We contrast five competing visions of the inference memory hierarchy and identify the regimes in which each is preferable, concluding with a heterogeneous "HBF-for-prefill, HBM-for-decode" fleet proposal and an agenda of open research problems (Sections 8–9).

(5) **Near-flash multiplier engines.** We design, verify and synthesise array, Wallace, Dadda and radix-4 Booth multipliers (4–16 bits, with ripple-carry or Kogge–Stone final adders, combinational and pipelined), an exact-accumulating FP8 MAC and a systolic MAC array for the HBF base die using an open-source flow in SkyWater 130 nm and ASAP7, compare them on every standard figure of merit with and without HBF, and quantify how many MACs a base die can host and what they buy at system level (Section 7).

2. Background and Related Work

2.1 The two phases of autoregressive inference

A request to an LLM proceeds in two phases. In **prefill**, the entire prompt of L tokens is processed in parallel: every linear layer performs a matrix–matrix product between an L-row activation matrix and the weight matrix, and the keys and values of every prompt token are written into the KV cache. In **decode**, the model generates output tokens one at a time; each step multiplies a single activation vector per sequence by every active weight matrix and attends over the full KV cache of that sequence. Batching B concurrent sequences turns decode's matrix–vector products into thin matrix–matrix products, raising arithmetic intensity proportionally to B, but each sequence's attention still reads its own KV cache, which does not amortise across the batch [4], [6].

The two phases therefore stress hardware differently: prefill is typically compute-bound and determines time-to-first-token (TTFT), whereas decode is memory-bound and determines time-per-output-token (TPOT). Production systems interleave them through iteration-level scheduling [7] and chunked prefill [8], or physically separate them onto different machine pools [9]–[11]. These system choices, as we show in Section 8, interact strongly with the choice of memory technology.

2.2 High Bandwidth Memory

HBM stacks DRAM dies on a base logic die, connected by thousands of TSVs and a 1024-bit (HBM3) or 2048-bit (HBM4) wide interface routed through a silicon interposer or bridge [23], [24]. Its key advantage is bandwidth density at the package shoreline; its limitations are capacity per stack, cost per bit, thermal density of the stack, and the limited "beachfront" around the accelerator die that caps the number of stacks — typically six to eight. Fine-grained DRAM organisations [25] and processing-in-memory variants [36] have been proposed to improve energy and effective bandwidth, but neither alters the fundamental capacity-per-stack limit of DRAM cells.

2.3 NAND flash and High Bandwidth Flash

3D NAND stores charge in vertically stacked cells formed in a single high-aspect-ratio etch [28]; contemporary parts exceed 200 word-line layers and store three (TLC) or four (QLC) bits per cell, yielding areal densities an order of magnitude above DRAM. Reading a page requires sensing a string through the vertical channel, which takes tens of microseconds (tR); programming takes hundreds of microseconds and erasure milliseconds. Reliability is managed by strong ECC (typically LDPC), read-retry, wear levelling and periodic refresh to counter retention loss and read disturb [27].

HBF, as described in public disclosures [29], [30], re-architects NAND for bandwidth rather than for storage density alone. Its principal ingredients are: (i) TSV-connected stacking of many NAND core dies above a logic base die, analogous to HBM; (ii) partitioning of each die into a large number of independently addressable sub-arrays or planes so that thousands of reads can proceed concurrently; (iii) a wide, HBM-like electrical interface so that the stack can sit on the same interposer as HBM; and (iv) a base die that hosts ECC, address translation and buffering. The public value proposition is HBM-class read bandwidth with 8–16× the capacity per stack — on the order of 512 GB for a first-generation stack — at similar cost, while accepting that write bandwidth and endurance are far lower than DRAM. Because specifications are not yet final, this paper treats every HBF parameter as an explicit, swept assumption (Table 2).

2.4 Related work

Weight offloading. FlexGen [12] and DeepSpeed-Inference [14] offload weights and KV to host DRAM and SSD, maximising throughput for latency-insensitive batch jobs; *LLM in a Flash* [13] exploits activation sparsity to run models larger than DRAM from smartphone flash. These systems traverse PCIe-class links (tens of GB/s) and thus cannot meet interactive TPOT targets for large models. Cambricon-LLM [31] couples an NPU with a flash chiplet with in-flash computation for on-device 70B inference, and in-storage attention offloading [32] moves attention into the SSD. HBF differs by placing flash *on the accelerator package* with two orders of magnitude more bandwidth.

Serving systems. PagedAttention/vLLM [6] manages the KV cache like virtual memory; Orca [7] introduced iteration-level scheduling; Sarathi-Serve [8] chunks prefills to bound decode stalls; DistServe [9], Splitwise [10] and Mooncake [11] disaggregate prefill and decode and treat the KV cache as a first-class, transferable object. **KV and weight compression** — multi-query and grouped-query attention [19], [20], multi-head latent attention [17], weight quantisation [39], [40] — reduce bytes moved and are complementary to HBF. **Memory disaggregation** through CXL [33], [34] extends capacity at DRAM latency but at link bandwidth well below HBM. **Processing near or in memory** [35], [36], including in-flash bulk operations [42], reduces data movement by computing at the memory. **SRAM-centric dataflow** machines [43] avoid off-chip memory on the critical path by distributing weights across many chips. To our knowledge no prior work has analysed on-package flash jointly with the prefill/decode split, MoE batch effects, endurance and disaggregated serving; this paper fills that gap.

3. Prefill and Decode Mechanics on a Tiered HBM–HBF Package

We model an accelerator package with effective compute throughput Π (FLOP/s), an HBM tier of capacity C_M and bandwidth β_M, and an HBF tier of capacity C_F and read bandwidth β_F. Model weights occupy W bytes (P parameters at b bytes each); a token's KV footprint is κ bytes. Each tier is accessed through independent channels, so transfers from the two tiers overlap.

3.1 Prefill

For a prompt chunk of L tokens processed against weights resident in tier X, the linear-layer FLOPs are approximately $2P_{act}L$ and the weight bytes read are $W_{touched}(L)$. The operational intensity with respect to weight traffic is therefore

$$I_{prefill} = 2\,P_{act}\,L \,/\, W_{touched}(L) \approx 2L / b \quad \textit{(dense models)} \tag{1}$$

which for FP8 weights ($b = 1$) equals $2L$ FLOP/byte. Prefill on tier X is compute-bound whenever I exceeds the ridge point Π/β_X. Solving for L gives the **minimum compute-bound chunk size**

$$L^{*} = \Pi \cdot W_{touched} \,/\, (2\,\beta_X\,P_{act}) \tag{2}$$

For a dense model $W_{\text{touched}} = W$ and $L^* = \Pi b/(2\beta_X)$, independent of model size. For an MoE model, a sufficiently long chunk touches every expert, so $W_{\text{touched}} \to W_{\text{total}}$ while P_{act} remains small; L^* is therefore inflated by the sparsity ratio P/P_{act}. This is the first mechanism by which MoE architectures and flash bandwidth interact: an MoE prefill on HBF needs thousands of tokens per weight pass to remain compute-bound (Section 6.4).

3.2 Decode

One decode iteration over a batch of B sequences with mean context S reads the touched weights once and each sequence's KV cache once. With weights in HBF and KV in HBM, the step time is

$$t_{step} = max(\ W_{touched}(B)/\beta_F\ ,\ B{\cdot}S{\cdot}\kappa/\beta_M\ ,\ F(B,S)/\Pi\ ) + t_{comm} \tag{3}$$

where $F(B,S) = B[2P_{\text{act}} + 4S{\cdot}d_{\text{attn}}{\cdot}n_{\text{layers}}]$ counts linear and attention FLOPs and t_{comm} captures collective latency when the model is sharded across N packages (we use $2n_{\text{layers}}{\cdot}\alpha$ with $\alpha = 10$ µs). In an HBM-only package both weight and KV traffic share β_M, so the first two terms add rather than overlap. Throughput per package is $B/(N{\cdot}t_{\text{step}})$, and the TPOT seen by each user is t_{step}. The batch is bounded by capacity:

$$B{\cdot}S{\cdot}\kappa\ /\ N \le \eta\ C_M, \quad W\ /\ N \le \eta\ C_F \ \ \textit{(hybrid);} \quad (W + B{\cdot}S{\cdot}\kappa)/N \le \eta\ C_M \ \ \textit{(HBM-only)} \tag{4}$$

with usable fraction $\eta = 0.9$. Equation (4) exposes the central trade-off: moving weights to HBF frees HBM capacity for KV — enlarging the feasible batch — while Equation (3) shows that it also moves weight traffic onto the slower flash channel.

3.3 Expert-touch probability in MoE decode

In an MoE layer with E experts and top-k routing, a single token reads only k experts. A batch of B tokens under approximately uniform routing touches each expert with probability

$$p_{touch}(B) = 1 - (1 - k/E)^B \tag{5}$$

so the expert bytes read per layer scale as $p_{\text{touch}}(B){\cdot}W_{\text{experts}}$. Fig. 3(b) plots Equation (5): for $E = 384$, $k = 8$ (a configuration representative of current trillion-parameter open models [44]), a batch of 32 already touches 49% of experts and a batch of 128 touches 93%. Consequently the decode weight traffic of a large MoE model is *not* a fixed $2P_{\text{act}}$ bytes per step: it grows almost linearly with batch until it saturates at the full expert set. On a flash tier with lower bandwidth than HBM, this traffic growth, rather than KV, becomes the binding constraint (Section 6.2).

3.4 Latency hiding and required concurrency

Flash read latency is three orders of magnitude above DRAM. By Little's law [38], sustaining bandwidth β with an access latency t_R requires Q outstanding reads of size s:

$$Q = \beta \cdot t_R / s \tag{6}$$

For $\beta = 0.8$ TB/s per stack, $t_R = 20$ µs and $s = 16$ KiB, $Q \approx 977$ concurrent page reads per stack (Fig. 6). This is the deepest architectural requirement of HBF: it is only an HBM-class device if the array is partitioned into on the order of a thousand independently operating read units, and if the base-die controller and the accelerator's DMA engines can keep that many requests in flight.

For dense models the weight access stream is entirely deterministic: the weights of layer ℓ+1 can be prefetched while layer ℓ computes. A decode step of 30–50 ms spread over 60–80 layers allots ≈0.5 ms per layer, which is 10–25× larger than t_R; a one-layer-ahead prefetch therefore hides flash latency completely. MoE layers break this determinism: which experts to fetch is known only after the router executes. If expert fetches are issued only then, each layer exposes one t_R, adding $n_{\text{layers}}{\cdot}t_R \approx 61 \times 20$ µs ≈ 1.2 ms (or 3.1 ms at 50 µs) per token — 2.4–6% of a 50 ms TPOT budget. Speculative expert prefetch

based on the previous layer's routing, or fetching all experts once the batch is large enough that $p_{\text{touch}} \approx 1$, removes this penalty.

4. Integrating HBF into Accelerator Systems

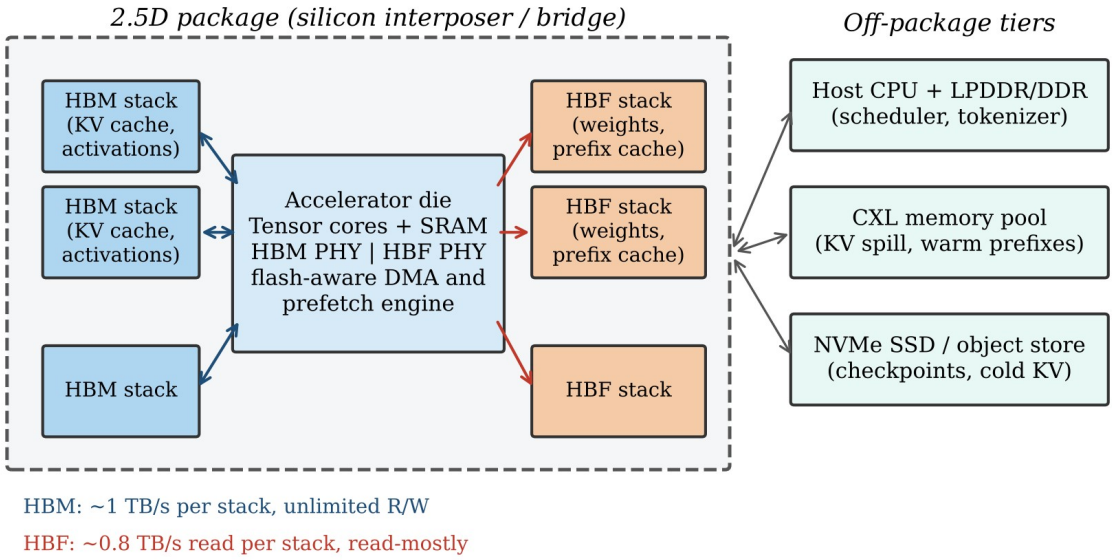


Fig. 1. Reference integration architecture. HBM and HBF stacks share the 2.5D interposer with the accelerator die, which integrates separate PHYs and a flash-aware DMA/prefetch engine. HBM holds the mutable KV cache and activations; HBF holds read-mostly model weights and write-once prefix caches. Host DRAM, CXL pools and SSDs form off-package tiers for spill, warm prefixes and checkpoints.

4.1 Physical integration

HBF is designed to share the interposer and, ideally, the PHY footprint of HBM, so that a package designer can trade HBM sites for HBF sites without redesigning the substrate. Three physical issues require attention. **Beachfront and stack height:** NAND core dies are thinner per bit than DRAM but a high-capacity stack may contain more dies; co-planarity with HBM stacks and the accelerator die constrains die thinning and TSV pitch. **Thermals:** NAND data retention degrades and raw bit-error rate rises with temperature, while the accelerator die adjacent to the stack can dissipate 1 kW. Placing HBF on the cooler edge of the interposer, adding thermal isolation or increasing ECC strength and refresh frequency are the available mitigations; this is an under-studied co-design problem. **Power delivery:** NAND programming requires high internal voltages generated by charge pumps; during bulk weight loading the program current of thousands of concurrently active planes must be budgeted against the package power envelope.

4.2 Controller and interface

A conventional SSD controller hides NAND behind a block interface with a page-mapped flash translation layer (FTL), garbage collection and deep request queues. HBF's base die must instead deliver deterministic, low-overhead *streaming reads* of large, aligned objects. We argue for a lightweight **read-optimised controller** with the following properties:

- **Region-mapped translation.** Weights are written once in large contiguous extents; a coarse region table (megabyte granularity) replaces the page-level FTL, eliminating mapping-table DRAM on the base die.

- **Load–seal–read life-cycle.** Regions are bulk-programmed when a model is loaded, sealed read-only and erased only when the model is replaced, so garbage collection and write amplification are near zero for the weight tier.
- **On-die LDPC with soft-decision fallback.** Hard-decision decoding at line rate on the base die keeps the common-case latency low; read-retry and soft decoding are exceptions handled out of band.
- **Read-disturb and retention management.** Weights are read billions of times; the controller must track per-block read counts and transparently relocate blocks that approach disturb limits — a background *write* source that must be included in the endurance budget.
- **Descriptor-based bulk DMA.** The accelerator issues descriptors of the form ⟨region, offset, length, destination⟩; the controller expands them into thousands of plane-level reads, satisfying Equation (6).

4.3 Data placement

Table 1 assigns each inference data object to a tier according to its access pattern, write rate, and latency sensitivity. The governing principle, which Section 6.5 quantifies, is that **HBF should hold only data that is written rarely and read in large, predictable streams**.

Table 1. Placement of inference data objects in a tiered HBM–HBF package.

Data object	Access pattern	Write frequency	Preferred tier	Rationale
Dense / shared weights	Sequential stream, every step	Per model update	HBF (or HBM if it fits)	Read-only; deterministic prefetch hides tR
MoE expert weights	Router-dependent, batch-dependent	Per model update	HBF; hot experts cached in HBM	Largest capacity consumer; sparsity tolerates flash bandwidth
KV cache (active)	Read every step, appended every token	Continuous	HBM	Mutable; would exhaust NAND endurance in days
Prefix / system-prompt KV	Read many times	Written once per prefix	HBF or CXL	Write-once-read-many; capacity-hungry
LoRA / adapters	Per-request selection	Rare	HBF, staged to HBM	Many variants, small each
Activations, workspace	Random, short-lived	Every layer	SRAM / HBM	Latency-critical and write-heavy
Checkpoints, cold KV	Bulk	Occasional	SSD / object store	Off-package; capacity only

4.4 Software stack

Integrating HBF changes four layers of the serving software. (1) **Model loader:** checkpoints must be laid out in HBF in the exact order the kernels consume them, with per-layer extents aligned to plane boundaries. (2) **Memory allocator:** the runtime must manage two physical address spaces with different granularity and mutability — a paged KV allocator [6] in HBM and an immutable region allocator in HBF. (3) **Compiler and kernels:** prefetch of layer ℓ+1 must be scheduled explicitly and overlapped with compute; for MoE, a speculative expert prefetcher keyed on router history is required. (4) **Scheduler:** the batch-size controller must be aware that in MoE decode the step time grows with B through Equation (5), not only through KV traffic, and must therefore choose B against the TPOT objective using a cost model of the form of Equation (3).

4.5 Reliability and endurance budget

The endurance-limited lifetime of an HBF stack of capacity C with rated P/E cycles N_{PE}, write amplification factor WAF and sustained write rate r is

$$T_{life} = N_{PE} \cdot C / (WAF \cdot r) \quad (7)$$

Section 6.5 evaluates Equation (7) for representative write sources. Because weights are written only when a model is deployed or updated, while the KV cache is written on every token, the two differ in write rate by three to four orders of magnitude — the quantitative basis of the placement in Table 1.

5. Evaluation Methodology

Because HBF silicon is not yet generally available, we evaluate it with an analytical model that implements Equations (2)–(7). Analytical roofline-style models are standard in early-stage architecture studies [4], [5] and, crucially, make every assumption explicit and reproducible. The model and figure scripts are released with this paper (see Data Availability). Table 2 lists device assumptions and Table 3 the workloads.

Table 2. Device and package assumptions (per package of eight memory sites).

Parameter	Value	Notes
Effective compute Π	1.5 PFLOP/s (FP8)	2.5 PFLOP/s peak × 0.6 utilisation
HBM stack	36 GB, 1.0 TB/s	12-high HBM3E class
HBF stack (baseline)	512 GB, 0.8 TB/s read	Projection; 80% of HBM bandwidth
HBF stack (optimistic)	512 GB, 1.6 TB/s read	Sensitivity study, Section 6.3
Configurations	HBM-8, HBM-6/HBF-2, HBM-4/HBF-4, HBM-2/HBF-6	Stacks of each type
Usable capacity η	0.9	Remainder: runtime, fragmentation
Collective latency α	10 µs per collective	Two collectives per layer when N > 1
NAND read latency tR	3–50 µs	Swept (Fig. 6)
P/E endurance	3k (TLC), 30k (pSLC)	pSLC uses one-third capacity; WAF = 2

Table 3. Workload models.

Model	Total params	Active params	Layers	Experts (E, k)	KV bytes/token
Dense-70B	70 B	70 B	80	—	160 KiB (GQA-8, FP8)
MoE-1T	1.0 T	32 B	61	384, 8	68.6 KiB (MLA, BF16)

Weights are stored in FP8. For each (model, configuration, N, S) we sweep the decode batch B and report the maximum throughput per package whose step time satisfies TPOT ≤ 50 ms; infeasible combinations (capacity violated at every B, or SLO violated at $B = 1$) are reported as such. Tensor/expert parallelism divides weight, KV and FLOP work evenly across N packages and adds t_{comm}; this idealisation favours scale-out and therefore makes the comparison conservative with respect to HBF. **Limitations:** the model omits energy, interconnect bandwidth contention, routing imbalance (Equation (5) assumes uniform routing; real routers are skewed, which reduces p_{touch} and favours HBF), ECC tail latency and queueing; it should be read as a bound-and-trend analysis to be validated against silicon when available.

6. Results

6.1 Roofline view

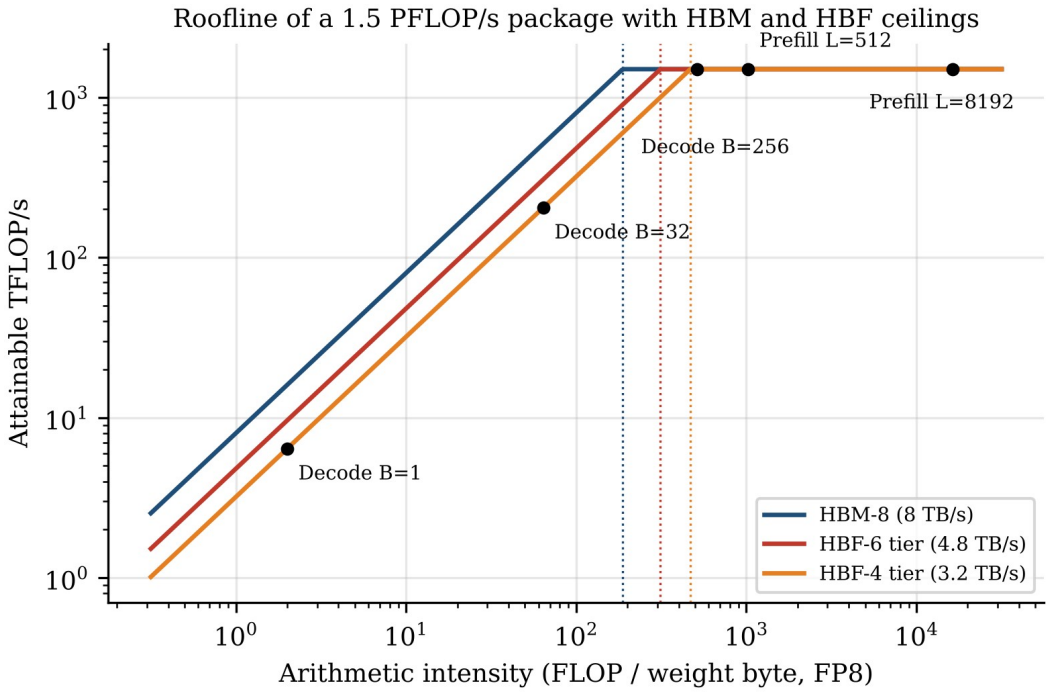


Fig. 2. Roofline of a 1.5 PFLOP/s package with an 8 TB/s HBM ceiling and 3.2/4.8 TB/s HBF ceilings (four or six HBF stacks). Decode at small batch lies deep in the bandwidth-bound region; prefill with chunks of a few hundred tokens or more is compute-bound even on the HBF ceiling.

Fig. 2 places prefill and decode on a roofline. The ridge point moves from 188 FLOP/byte for an all-HBM package to 469 FLOP/byte when weights stream from four HBF stacks. Decode with $B \le 32$ operates at 2–64 FLOP/byte and is bandwidth-bound on any tier, so every byte/s of weight bandwidth translates directly into tokens/s. Dense prefill, at $2L$ FLOP/byte, crosses the HBF ridge for $L \ge 235$ and is thereafter insensitive to the memory technology. The roofline thus predicts the qualitative split developed below: **HBF is almost free for prefill and expensive for latency-bound decode**.

6.2 Decode under a latency objective

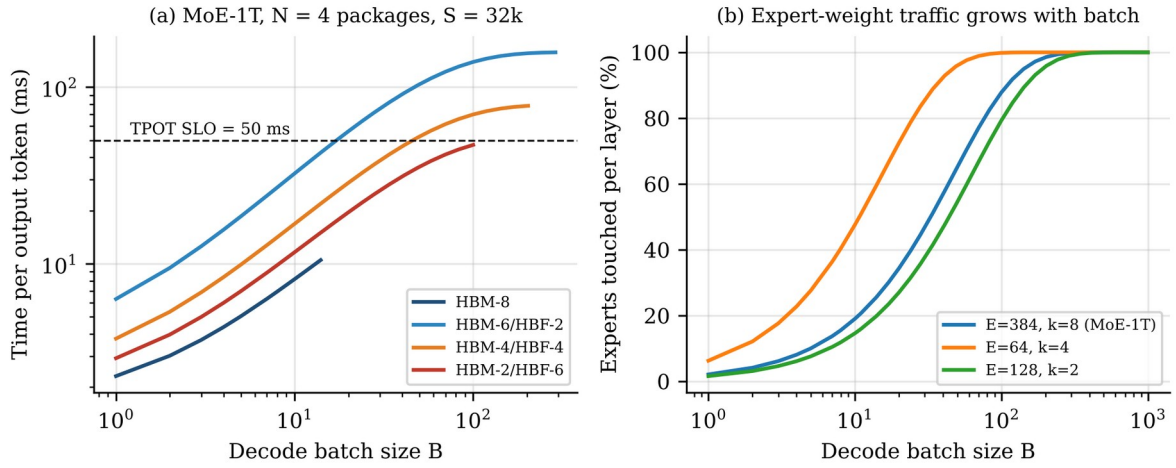


Fig. 3. (a) TPOT versus decode batch for MoE-1T on N = 4 packages at 32k context. The HBM-8 curve terminates at B = 14 because weights plus KV exhaust HBM; hybrid configurations sustain larger batches but their step time

rises with B because more experts are touched. (b) Expert-touch probability, Equation (5), for three MoE geometries.

Dense-70B. A 70B model in FP8 fits in a single HBM-8 package, and HBF never helps. At 8k context and $N = 1$, HBM-8 delivers 4,308 tokens/s per package within the SLO versus 2,980 for HBM-4/HBF-4 and 1,490 for HBM-2/HBF-6; the pattern persists for every N and at 32k context (1,077 vs. 745 vs. 373). Replacing HBM stacks with HBF removes KV capacity and KV bandwidth that a model of this size needs more than it needs weight capacity. This negative result is important for the debate: **HBF is not a general-purpose HBM substitute**.

MoE-1T. The picture changes for a trillion-parameter MoE (Fig. 3, Fig. 4, Table 4). An HBM-only deployment is infeasible below four packages. With HBF the model fits in a *single* package, which serves 152 (HBM-4/HBF-4) to 248 (HBM-2/HBF-6) tokens/s at 46–49 ms TPOT. At four packages and 32k context, HBM-8 is capacity-starved — after storing 250 GB of weights per package only ≈9 GB remains for KV, limiting the batch to 12 — and delivers 321 tokens/s per package, while HBM-2/HBF-6 sustains a batch of 108 and 562 tokens/s per package, a **1.75× advantage**. At 8k context the two are at parity (574 vs. 562) because shorter contexts relieve HBM capacity. From eight packages upward, the aggregate HBM capacity of the HBM-only design is sufficient and its higher bandwidth wins (1,645 vs. 1,307 tokens/s per package at 32k).

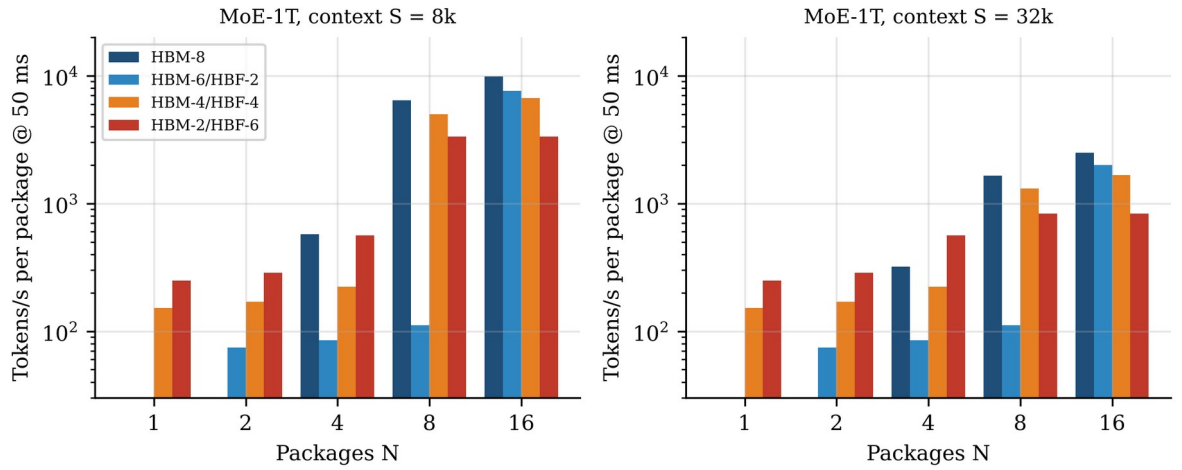


Fig. 4. Best decode throughput per package subject to TPOT ≤ 50 ms for MoE-1T versus the number of packages N, at 8k and 32k context. Missing bars denote infeasible configurations. HBF configurations are the only feasible options at N ≤ 2 and lead at N = 4 for long contexts; HBM-only leads once N is large enough to hold weights plus KV in HBM.

Table 4. Decode throughput per package (tokens/s) for MoE-1T at TPOT ≤ 50 ms, 32k context. Batch size in parentheses; the binding resource is the tier listed in the last column for the best hybrid entry.

Packages N	HBM-8	HBM-6/HBF-2	HBM-4/HBF-4	HBM-2/HBF-6	Binding resource
1	infeasible	infeasible	152 (7)	248 (12)	HBF weight bandwidth
2	infeasible	74 (7)	169 (16)	287 (27)	HBF weight bandwidth
4	321 (12)	85 (16)	223 (43)	562 (108)	HBF weight bandwidth
8	1,645 (421)	112 (43)	1,307 (421)	835 (214)	HBF / HBM-KV
16	2,500 (1,289)	2,000 (1,289)	1,668 (824)	835 (421)	HBM-KV bandwidth

Two further observations matter for designers. First, **HBF stack count should be sized by bandwidth, not by capacity**: HBM-6/HBF-2 provides 1 TB of flash — enough to hold the model — but its 1.6 TB/s of weight bandwidth makes it the worst configuration at every $N \leq 8$. Second, the binding resource in every capacity-constrained hybrid case is HBF weight bandwidth driven by the expert-touch growth of Equation (5), not KV traffic. Techniques that reduce p_{touch} — expert-affinity batching, hot-expert caching in HBM, or routing-aware request grouping — therefore have outsized value on HBF systems.

6.3 Sensitivity to HBF bandwidth

Doubling HBF read bandwidth to 1.6 TB/s per stack — plausible for later generations with more sub-array parallelism or a wider interface — raises single-package MoE-1T throughput from 152 to 348 tokens/s (HBM-4/HBF-4) and from 248 to 589 tokens/s (HBM-2/HBF-6). At $N = 4$ and 32k context, HBM-4/HBF-4 reaches 1,342 tokens/s per package with a batch of 214, **4.2×** the HBM-8 result, and HBM-2/HBF-6 becomes HBM-KV-bound at 836 tokens/s. The benefit of HBF is thus highly elastic in its bandwidth: at parity with HBM bandwidth the hybrid would dominate every capacity-constrained regime, whereas at half of HBM bandwidth its advantage would vanish except for feasibility at small N.

6.4 Prefill

Table 5. Minimum compute-bound prefill chunk L* from Equation (2) and time to prefill an 8,192-token prompt (compute-limited and weight-traffic-limited components).

Model / configuration	L* (tokens)	t_compute (s)	t_weights (s)	Bound at L = 8,192
Dense-70B / HBM-8	94	0.82	0.009	Compute
Dense-70B / HBM-4/HBF-4	234	0.82	0.022	Compute
MoE-1T / HBM-8 (N ≥ 4)	2,930	0.44	0.125	Compute
MoE-1T / HBM-4/HBF-4	7,324	0.44	0.313	Compute (marginal)
MoE-1T / HBM-2/HBF-6	4,883	0.44	0.208	Compute

Table 5 confirms that dense prefill is compute-bound on HBF for any chunk above a few hundred tokens. MoE prefill is more demanding: with four HBF stacks a chunk must contain at least 7,324 tokens for flash weight traffic to be hidden behind computation, because a long chunk touches every expert while only 32B parameters are active per token. Chunked-prefill schedulers [8] that use 512–2,048-token chunks to bound decode interference would, on an HBF-weight package serving a large MoE, become weight-bandwidth-bound. This creates a direct tension between chunked prefill and HBF that **prefill–decode disaggregation resolves**: a dedicated prefill node can batch prompts into large chunks without harming any decode TPOT.

6.5 Endurance

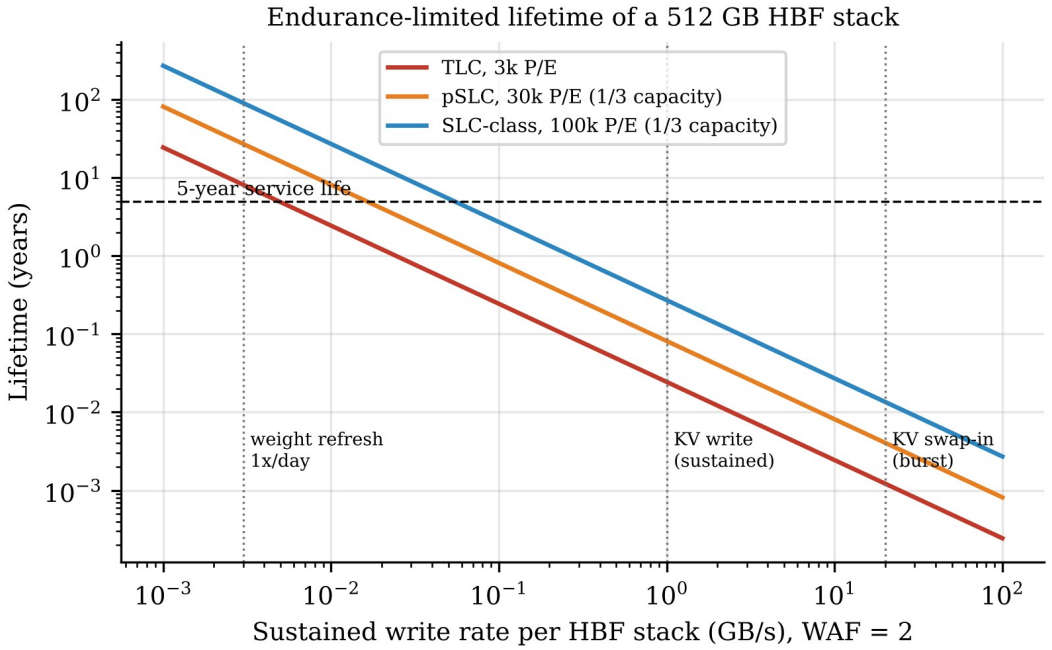


Fig. 5. Endurance-limited lifetime of a 512 GB HBF stack versus sustained write rate (WAF = 2). Vertical markers indicate a daily full rewrite of a 256 GB weight shard, sustained KV writes (≈1 GB/s) and KV swap-in bursts (≈20 GB/s).

Fig. 5 evaluates Equation (7). Rewriting a stack's 256 GB weight shard once per day (0.003 GB/s) consumes the endurance of a TLC stack in 2,963 days (8.1 years) and of a pSLC configuration in 27 years — comfortably beyond a five-year service life even with daily model updates. In contrast, a sustained KV write rate of 1 GB/s — typical of a package generating several thousand tokens/s with 70–160 KiB of KV per token — wears out TLC in **8.9 days** and pSLC in 30 days; KV swap-in bursts at 20 GB/s would do so in hours. The KV cache must therefore never be placed in HBF as a general-purpose spill target. A write-once prefix cache — a system prompt or shared document reused by many requests [11] — has a write rate set by the arrival of *new* prefixes and is acceptable when that rate is below ≈20 MB/s per stack.

6.6 Concurrency requirements

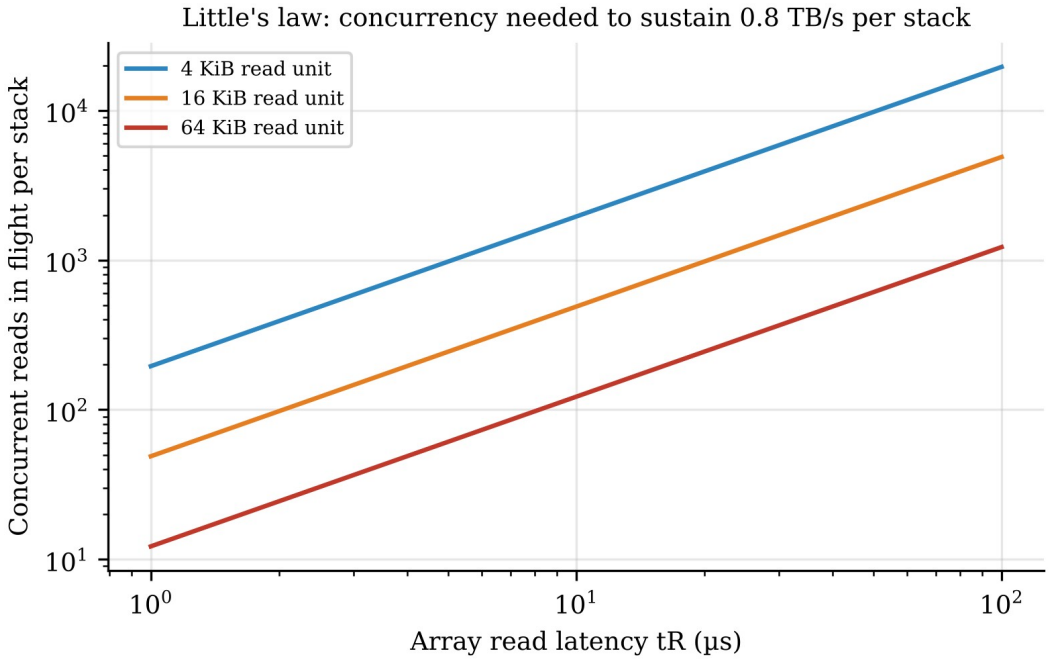


Fig. 6. Concurrent reads in flight per stack needed to sustain 0.8 TB/s, from Equation (6), for 4, 16 and 64 KiB read units.

Fig. 6 translates the bandwidth target into a parallelism requirement. With 16 KiB pages, a stack must keep 146 reads in flight at t_R = 3 µs (low-latency SLC-class arrays), 977 at 20 µs and 2,441 at 50 µs; with 4 KiB units these numbers quadruple to 586–9,766. A 16-die stack with four planes per die offers only 64 conventional planes, more than an order of magnitude short of the 20 µs requirement. Hence HBF requires either many more independently sensed sub-arrays per die, larger effective read units through multi-plane reads, lower-latency array designs, or all three — a central VLSI and circuit-design challenge for the technology.

7. Near-Flash Compute: Integrating Array Multiplier Engines into the HBF Base Die

Section 6.2 identified HBF weight bandwidth as the binding resource whenever HBF is used for capacity, and the architecture debate of Section 8 (Position D) argues that computing *inside* the stack would remove weight traffic from the interposer. This section makes that argument concrete at the circuit level. We ask four questions: (i) which multiplier architecture is best suited to a near-flash engine; (ii) how the candidates compare as stand-alone arithmetic units, i.e. *without* HBF; (iii) how many multiply–accumulate (MAC) units an HBF base die can host within realistic area and thermal budgets; and (iv) what the system gains in energy and decode throughput relative to a conventional HBM system and to HBF with far compute.

7.1 Placement options

Table 6. Candidate locations for multiplier hardware in an HBF stack.

Location	Mechanism	Precision	Main obstacle	Assessment
Logic base die	Weights stream from page buffers through TSVs into a digital MAC array	INT4/INT8/ FP8, exact	Area and heat beneath NAND dies	Feasible; studied here

Location	Mechanism	Precision	Main obstacle	Assessment
Page-buffer level	Bit-serial shift/add/bitwise logic in each plane's page buffer [42]	Low; bit-serial	Hundreds of cycles per multiply	Niche (binary/ternary)
In NAND array (analog)	Conductance-weighted string-current summation	≈4–6 bit, noisy	ADCs, drift, read disturb	Not yet viable for LLMs

7.2 Multiplier architectures and analytical complexity

All candidates share the same partial-product generation for signed operands, the modified Baugh–Wooley (BW) formulation [45], [60], which avoids sign extension by complementing the partial products that involve exactly one sign bit and adding two constants:

$$A{\cdot}B = a_{n-1}b_{n-1}2^{2n-2} + \Sigma_{i,j<n-1}\, a_i b_j 2^{i+j} + \Sigma_{i<n-1}\, \neg(a_i b_{n-1}) 2^{i+n-1} + \Sigma_{j<n-1}\, \neg(a_{n-1} b_j) 2^{j+n-1} + 2^n + 2^{2n-1} \quad (8)$$

The architectures differ in how they reduce the n^2-bit partial-product matrix to two rows and in the final carry-propagate adder (CPA). The **array multiplier** accumulates one row per stage with a carry-save adder line, giving a regular, abutting layout but a reduction depth that grows linearly with n. **Wallace** [46] and **Dadda** [47] trees reduce all columns concurrently with 3:2 counters; Dadda uses the minimum number of counters per level and the height sequence $d_{j+1} = \lfloor 1.5\, d_j \rfloor$ (2, 3, 4, 6, 9, 13, …). **Radix-4 Booth** recoding [48], [49] of B halves the number of partial-product rows to $n/2$ at the cost of a recoder and a 5:1 selector per bit; we reduce Booth rows with a Dadda tree and handle sign extension with the same complement-and-constant technique as Equation (8). Each reduction can terminate in a ripple-carry adder (RCA) or a Kogge–Stone (KS) parallel-prefix adder [50]. Table 7 lists the counter counts and logic depths measured on the generated netlists (2-input-gate levels; a full adder counts two levels), which follow the closed forms

$$\textit{Array:}\ \ N_{FA} = (n-1)^2,\ \ N_{HA} = n,\ \ D_{tree} = 2(n-2) + 1 \quad (9a)$$

$$\textit{Dadda:}\ \ D_{tree} = 2L + 1,\ \ L = \textit{smallest } j \textit{ with } d_j \geq n \quad (9b)$$

— for example, a 16-bit array needs a 29-level reduction while a 16-bit Dadda tree needs 13 levels, and Booth recoding reduces the 16-bit counter count from 225 to 120 full adders.

Table 7. Analytical structure of the generated multipliers (counts from the netlist generator; depth in 2-input-gate levels).

Design	Full adders	Half adders	Reduction depth	Total depth	Pipeline flops
Array 4×4	9	4	5	13	—
Dadda 4×4	9	4	5	13	—
Booth-4 4×4	6	6	7	18	—
Array 8×8	49	8	13	29	—
Wallace 8×8	47	28	9	29	—
Dadda 8×8	49	8	9	29	—
Dadda+KS 8×8	36	6	9	19	—
Booth-4 8×8	28	12	11	34	—
Booth-4+KS 8×8	15	9	11	21	—
Array 16×16	225	16	29	61	—
Wallace 16×16	221	81	13	60	—
Dadda 16×16	225	16	13	61	—
Dadda+KS 16×16	196	14	13	25	—
Booth-4 16×16	120	24	13	66	—

Design	Full adders	Half adders	Reduction depth	Total depth	Pipeline flops
Booth-4+KS 16×16	91	21	13	25	—
Array 8×8, 3-stage	49	8	5	15	95
Dadda 8×8, 2-stage	49	8	4	15	72
Booth 8×8, 2-stage	28	12	4	15	64
Array 16×16, 4-stage	225	16	7	31	263

With an RCA, the 2n-bit carry chain dominates the total depth of every tree multiplier (29 levels at n = 8 for array, Wallace and Dadda alike); the tree advantage only becomes visible when it is paired with a parallel-prefix CPA (19 levels for Dadda+KS at n = 8). Pipelining cuts the per-stage depth to 15 levels at 8 bits and 31 at 16 bits.

7.3 RTL implementation and verification

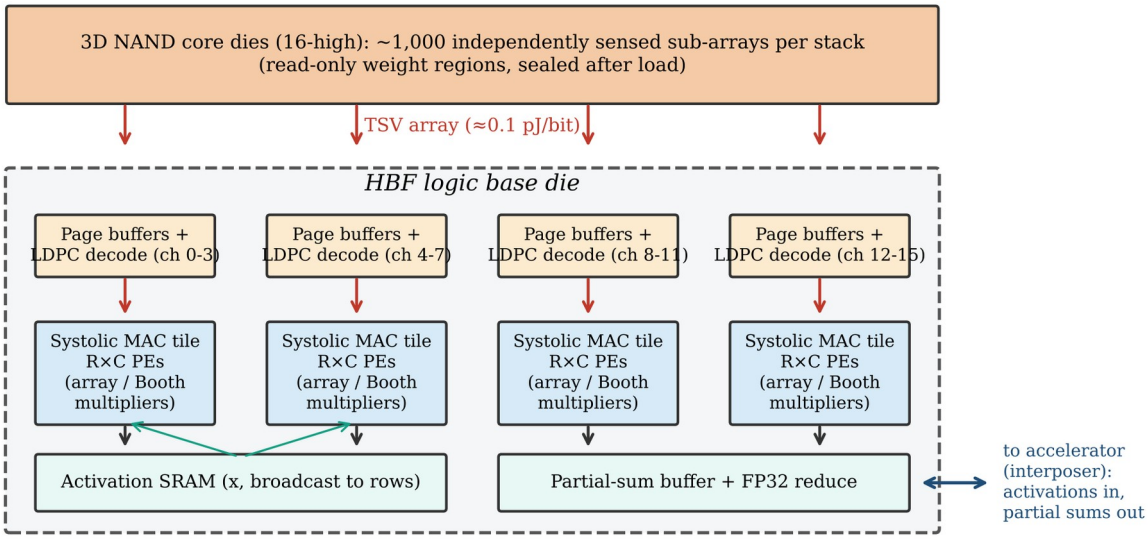


Fig. 7. Proposed near-flash MAC engine in the HBF logic base die. Each group of NAND channels delivers decoded weight pages to a systolic MAC tile; activations are broadcast from a small SRAM, and only activations and partial sums cross the interposer to the accelerator.

All multipliers are produced by an open-source Python generator that emits structural Verilog from the partial-product dot matrix, so that every architecture shares identical partial-product logic and differs only in the reduction and CPA. The generator supports arbitrary widths, signed (BW or Booth) and unsigned operands, and the insertion of pipeline registers after any reduction stage or at the midpoint of the CPA. On top of the multipliers we implement three engines for the base die:

- **INT8 MAC processing element (PE):** BW array multiplier, one product register and a 32-bit accumulator, with pass-through registers for the systolic dataflow.
- **FP8 (E4M3) [57] MAC with exact accumulation [58]:** a 4×4 unsigned array multiplier on the significands (with the hidden bit), a 5-bit exponent adder, a barrel shifter that aligns the 8-bit significand product to a fixed-point grid with resolution 2^{-18}, and a 48-bit two's-complement accumulator. Because every E4M3 product is an integer multiple of 2^{-18} with magnitude below 2^{18}, Equation (10) is exact and a dot product of up to 2^{11} full-scale terms cannot overflow:

- **4×8 output-stationary systolic array:** each column is fed by one NAND channel lane (one weight byte per cycle), each row holds one sequence of the decode batch; PE(*r*, *c*) accumulates $\Sigma_k A_{rk} W_{kc}$.

$$w \cdot x = (-1)^s \cdot m_w m_x \cdot 2^{Ew+Ex-20} = (-1)^s \cdot [\, m_w m_x \ll (E_w + E_x - 2)\,] \cdot 2^{-18} \tag{10}$$

with $s = s_w \oplus s_x$, significands $m = \{e \neq 0, \text{mantissa}\}$ and effective exponents $E = \max(e, 1)$.

Functional verification used Icarus Verilog [56]. Every 4- and 8-bit multiplier was checked **exhaustively** (all 256 or 65,536 operand pairs), 16-bit designs with 300,000 random vectors plus all sign/extreme corner combinations, pipelined variants cycle-accurately against a latency-aligned reference, the FP8 MAC exhaustively for all 65,536 products and on 200 random dot products of length 1–1,024 against an exact rational (Python `Fraction`) reference, and the systolic array on 50 random 4×64 by 64×8 matrix products. All 26 test suites pass with zero mismatches.

7.4 Synthesis and power methodology

Each design was synthesised with Yosys [52] and ABC [53] to two open standard-cell libraries representing the range of plausible base-die processes: **SkyWater 130 nm** [55] (sky130_fd_sc_hd, TT, 1.8 V, 25 °C), standing in for a mature, low-cost logic node, and the **ASAP7** 7 nm predictive library [54] (RVT, TT, 0.7 V), standing in for an advanced node. Every design was mapped twice — *area-driven* (ABC &nf without a delay target) and *delay-driven* (minimum delay target followed by buffering and gate sizing); adders inferred from behavioural code are mapped by Yosys to Brent–Kung parallel-prefix structures [51]. The critical path is ABC's static timing estimate; for registered metrics we add the library flip-flop clock-to-Q and setup times (637 ps for sky130 and 103 ps for ASAP7). Power was obtained from **unit-delay gate-level simulation** of the mapped netlists — which, unlike zero-delay simulation, propagates the spurious transitions (glitches) characteristic of array multipliers — over 3,000 cycles (300 for the 17,000-cell systolic array) for two operand statistics: uniformly random operands and a zero-mean Gaussian distribution with σ = 1/8 of full scale that resembles quantised LLM weights. Switching energy is $\tfrac{1}{2}C \cdot V^2$ per toggle with C the sum of fan-out pin capacitances from the liberty file, internal energy is taken from the liberty internal_power tables, and leakage from the cell leakage attributes. The figures of merit are defined as

$$T_{clk} = t_{comb} + t_{clk \to Q} + t_{setup}, \quad E_{op} = (E_{cycle} + P_{leak} T_{clk}) / ops \tag{11a}$$

$$ADP = A \cdot T_{clk}, \quad EDP = E_{op} \cdot T_{clk}, \quad \eta = 1 / E_{op}, \quad \rho = ops / (A \cdot T_{clk}) \tag{11b}$$

where η is energy efficiency (TOPS/W, counting one MAC as one operation) and ρ is throughput density (GOPS/mm²). These are pre-layout results: wire capacitance and clock-tree power are excluded, which lowers absolute energy by an amount that is similar across architectures, so *relative* comparisons are more reliable than absolute values.

7.5 Stand-alone multiplier comparison (without HBF)

We first compare the multipliers as generic arithmetic units, independent of where they are deployed. Tables 8 and 9 report every figure of merit for the 8-bit designs in both libraries and both synthesis modes, and Fig. 8 normalises the main metrics to the array multiplier.

Table 8. Signed 8×8 multipliers in SkyWater 130 nm (1.8 V). † = delay-driven synthesis (otherwise area-driven); KS = Kogge–Stone final adder. E/op includes leakage over one registered cycle; Gauss./unif. = operand statistics.

Design	Area (µm²)	Cells	T_clk (ps)	f_max (GHz)	E/op Gauss. (fJ)	E/op unif. (fJ)	Leak (µW)	ADP (µm²·ns)	EDP (fJ·ns)	GOPS /mm²	TOPS / W
Array (BW)	1,897	221	4,856	0.21	699	667	0.001	9,210	3,396	109	1.43
Array (BW) †	1,943	221	4,444	0.23	700	668	0.001	8,634	3,110	116	1.43

Design	Area (μm²)	Cells	T_clk (ps)	f_max (GHz)	E/op Gauss. (fJ)	E/op unif. (fJ)	Leak (μW)	ADP (μm²·ns)	EDP (fJ·ns)	GOPS /mm²	TOPS / W
Wallace	2,021	249	4,534	0.22	643	648	0.001	9,163	2,915	109	1.56
Wallace †	2,057	250	4,041	0.25	635	639	0.001	8,312	2,565	120	1.58
Dadda	1,866	254	4,265	0.23	752	708	0.001	7,956	3,207	126	1.33
Dadda †	1,932	254	3,788	0.26	781	737	0.001	7,318	2,958	137	1.28
Dadda + KS	1,859	248	4,218	0.24	775	728	0.001	7,842	3,268	128	1.29
Dadda + KS †	1,932	248	3,840	0.26	778	731	0.001	7,418	2,988	135	1.29
Booth-4	1,493	200	5,040	0.20	806	762	0.000	7,523	4,060	133	1.24
Booth-4 †	1,610	202	4,088	0.24	889	853	0.001	6,584	3,634	152	1.13
Booth-4 + KS	1,647	209	3,680	0.27	732	722	0.001	6,060	2,694	165	1.37
Booth-4 + KS †	1,813	209	3,234	0.31	864	852	0.001	5,864	2,795	171	1.16
Tool a×b	2,380	294	4,359	0.23	526	529	0.001	10,373	2,294	96	1.90
Tool a×b †	2,415	295	3,849	0.26	534	541	0.001	9,293	2,056	108	1.87
Array, 3-stage	3,384	359	3,075	0.33	5,927	6,007	0.001	10,407	18,226	96	0.169
Array, 3-stage †	3,392	359	2,690	0.37	5,931	6,012	0.001	9,123	15,952	110	0.169
Dadda, 2-stage	3,157	311	3,079	0.32	5,246	5,334	0.001	9,721	16,153	103	0.191
Dadda, 2-stage †	3,171	311	2,737	0.37	5,249	5,338	0.001	8,678	14,368	115	0.191
Booth-4, 2-stage	2,775	262	3,079	0.32	5,309	5,411	0.001	8,546	16,347	117	0.188
Booth-4, 2-stage †	2,785	262	2,703	0.37	5,318	5,420	0.001	7,528	14,373	133	0.188

Table 9. Signed 8×8 multipliers in ASAP7 (0.7 V); notation as in Table 8.

Design	Area (μm²)	Cells	T_clk (ps)	f_max (GHz)	E/op Gauss. (fJ)	E/op unif. (fJ)	Leak (μW)	ADP (μm²·ns)	EDP (fJ·ns)	GOPS /mm²	TOPS / W
Array (BW)	24.4	249	695	1.44	17.9	16.9	0.012	17.0	12.5	58,846	55.8
Array (BW) †	25.4	249	644	1.55	19.1	18.0	0.013	16.4	12.3	61,108	52.3
Wallace	27.9	315	592	1.69	16.8	16.5	0.013	16.5	9.96	60,566	59.4
Wallace †	30.2	315	540	1.85	21.4	21.2	0.015	16.3	11.6	61,350	46.6
Dadda	24.0	254	549	1.82	18.0	17.5	0.011	13.2	9.91	75,848	55.4
Dadda †	25.5	254	507	1.97	22.1	21.6	0.013	12.9	11.2	77,370	45.2
Dadda + KS	24.1	256	578	1.73	17.5	17.0	0.011	13.9	10.1	71,720	57.3
Dadda + KS †	26.3	256	517	1.93	23.8	23.3	0.014	13.6	12.3	73,431	42.1
Booth-4	21.2	215	571	1.75	15.8	15.5	0.010	12.1	9.04	82,405	63.2
Booth-4 †	21.8	215	528	1.90	17.9	17.6	0.011	11.5	9.47	86,949	55.7
Booth-4 + KS	21.6	214	537	1.86	15.7	15.4	0.010	11.6	8.42	86,137	63.8
Booth-4 + KS †	22.4	214	482	2.07	17.2	17.1	0.011	10.8	8.31	92,457	58.0
Tool a×b	31.3	331	598	1.67	18.7	18.4	0.014	18.7	11.1	53,521	53.6
Tool a×b †	34.6	331	512	1.95	23.3	23.1	0.018	17.7	11.9	56,438	42.9
Array, 3-stage	50.7	356	260	3.85	93.0	94.0	0.023	13.2	24.1	76,017	10.8
Array, 3-stage †	50.8	356	252	3.97	92.8	93.8	0.022	12.8	23.4	78,104	10.8
Dadda, 2-stage	46.2	333	311	3.21	86.1	87.4	0.021	14.4	26.8	69,557	11.6
Dadda, 2-stage †	46.8	333	295	3.39	86.0	87.3	0.022	13.8	25.4	72,474	11.6
Booth-4, 2-stage	40.4	281	316	3.17	83.1	86.9	0.018	12.8	26.2	78,366	12.0
Booth-4, 2-stage †	40.6	281	314	3.19	83.1	86.8	0.018	12.7	26.1	78,541	12.0

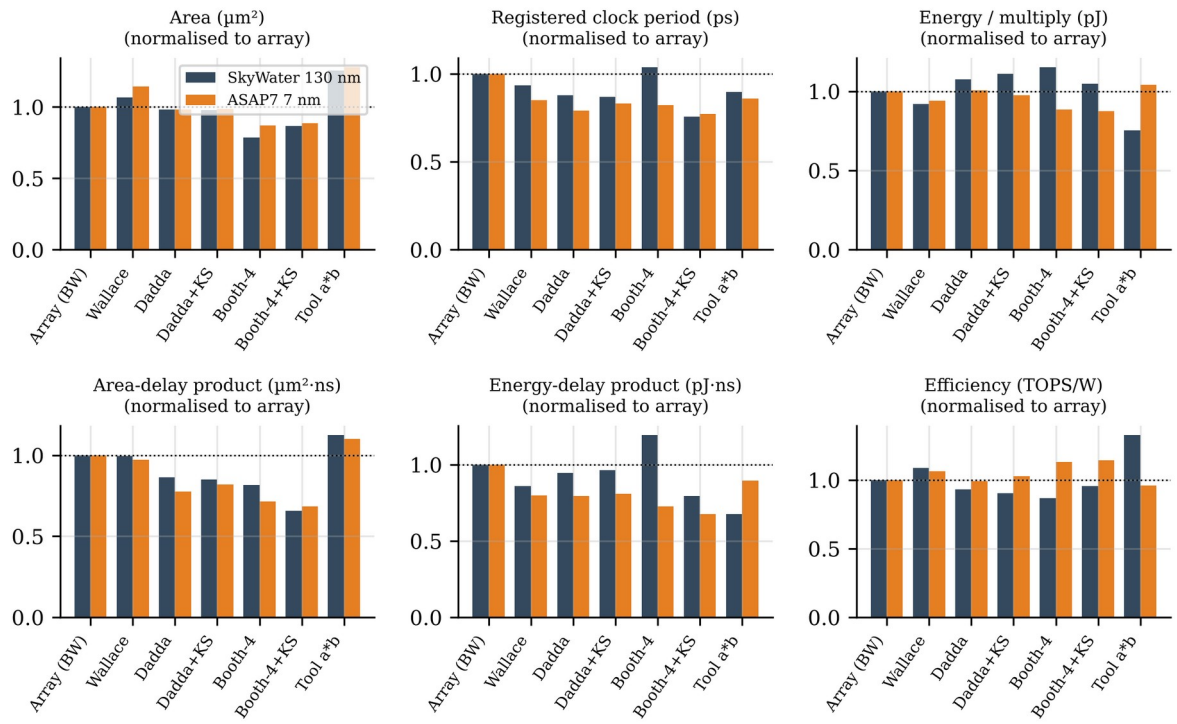


Fig. 8. Six figures of merit for the 8-bit multiplier architectures in both libraries, normalised to the Baugh–Wooley array multiplier (area-driven synthesis, Gaussian operands). Values below 1 are better except for TOPS/W.

Area. The smallest 8-bit multiplier is the Radix-4 Booth + Dadda in ASAP7 (21.2 µm²) and the Radix-4 Booth + Dadda in sky130 (1,493 µm²). The array multiplier occupies 24.4 µm² (ASAP7) and 1,897 µm² (sky130); Booth recoding reduces area by 13% in ASAP7 and reduces it by 21% in sky130: the saving of 21 full adders (Table 7) outweighs the recoder and 5:1 selector logic. **Speed.** The fastest registered design is the Booth-4 + Dadda + Kogge-Stone CPA (537 ps, 1.86 GHz) in ASAP7 and the Booth-4 + Dadda + Kogge-Stone CPA (3,680 ps) in sky130, against 695 ps and 4,856 ps for the array. **Energy.** The lowest energy per multiply is achieved by the Booth-4 + Dadda + Kogge-Stone CPA in ASAP7 (15.7 fJ) and the Tool default (a×b) in sky130 (0.526 pJ); the array multiplier needs 17.9 fJ and 0.699 pJ respectively. The tool-default implementation of a×b (Yosys/ABC) is larger than the hand-structured array (31.3 vs. 24.4 µm²) and faster (598 vs. 695 ps), which shows that a structural description still matters for arithmetic in an open-source flow.

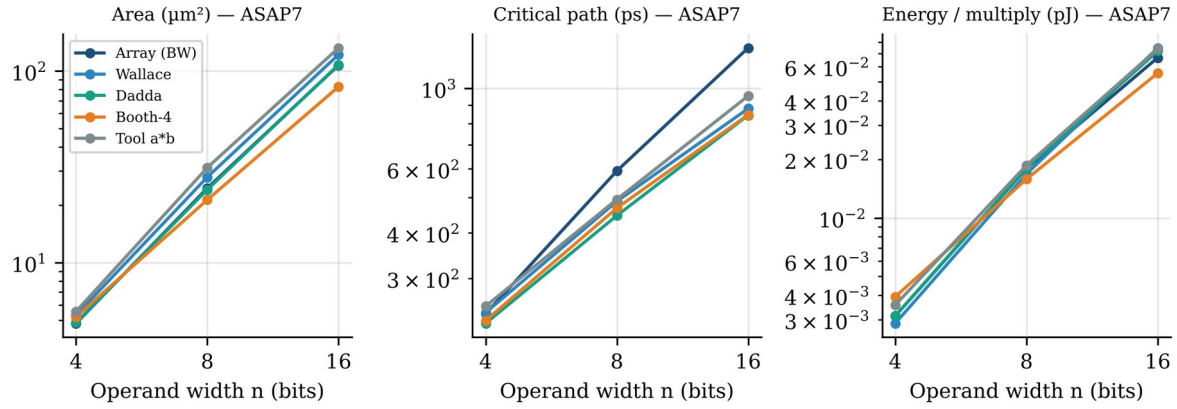


Fig. 9. Scaling of area, critical path and energy with operand width (4, 8, 16 bits) in ASAP7.

Width scaling (Fig. 9, Table 10). Area and energy grow roughly quadratically with *n*: the ASAP7 array multiplier grows from 4.8 µm² at 4 bits to 24.4 and 107.1 µm² at 8 and 16 bits. The critical path of

the array grows linearly (240, 592, 1,291 ps), whereas tree multipliers with a parallel-prefix CPA grow logarithmically: at 16 bits Dadda+KS reaches 676 ps and Booth+KS 595 ps, i.e. 2.17× faster than the array. At 16 bits Booth recoding also pays off in energy (55.5 fJ vs. 66.7 fJ for the array). For the 4- and 8-bit operands that dominate LLM inference, however, the array multiplier remains within 33% of the best design on area, clock period and energy in both libraries while being the most regular to lay out — a property that matters on a base die whose routing resources are shared with thousands of TSVs. Fig. 10 shows the complete energy–delay design space including both synthesis modes: delay-driven mapping shortens the clock period by 7–19% at up to 36% more energy per multiply, and the pipelined designs occupy a separate high-frequency, high-energy region.

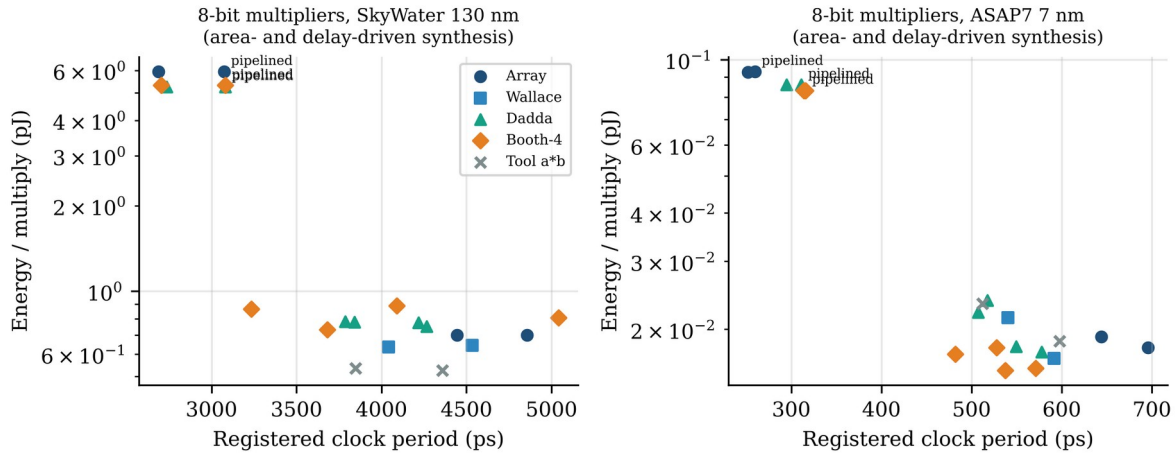


Fig. 10. Energy–delay design space of all 8-bit multipliers (both synthesis modes; pipelined variants annotated). Points toward the lower left are Pareto-better.

Table 10. Width sweep (4, 8, 16 bits) for all architectures in both libraries (area-driven synthesis, Gaussian operands). EDP in fJ·ns.

Architecture	n	Library	Area (μm²)	t_comb (ps)	f_max (GHz)	E/op (fJ)	EDP (fJ·ns)	TOPS/W
Array (BW)	4	sky130	370	2,307	0.34	116	342	8.60
Array (BW)	4	ASAP7	4.8	240	2.92	3.13	1.07	319
Array (BW)	8	sky130	1,897	4,218	0.21	699	3,396	1.43
Array (BW)	8	ASAP7	24.4	592	1.44	17.9	12.5	55.8
Array (BW)	16	sky130	8,506	8,836	0.11	2,724	25,808	0.367
Array (BW)	16	ASAP7	107	1,291	0.72	66.7	93.0	15.0
Wallace	4	sky130	387	2,063	0.37	114	307	8.78
Wallace	4	ASAP7	5.3	241	2.90	2.87	0.989	348
Wallace	8	sky130	2,021	3,897	0.22	643	2,915	1.56
Wallace	8	ASAP7	27.9	488	1.69	16.8	9.96	59.4
Wallace	16	sky130	8,562	7,210	0.13	2,609	20,472	0.383
Wallace	16	ASAP7	122	879	1.02	74.8	73.5	13.4
Dadda	4	sky130	348	2,060	0.37	163	441	6.12
Dadda	4	ASAP7	4.9	225	3.04	3.13	1.03	319
Dadda	8	sky130	1,866	3,627	0.23	752	3,207	1.33
Dadda	8	ASAP7	24.0	446	1.82	18.0	9.91	55.4
Dadda	16	sky130	7,734	6,779	0.13	2,743	20,341	0.365
Dadda	16	ASAP7	108	843	1.06	72.1	68.2	13.9
Booth-4	4	sky130	360	2,303	0.34	180	530	5.55
Booth-4	4	ASAP7	5.2	230	3.01	3.93	1.31	254

Architecture	n	Library	Area (μm²)	t_comb (ps)	f_max (GHz)	E/op (fJ)	EDP (fJ·ns)	TOPS/W
Booth-4	8	sky130	1,493	4,403	0.20	806	4,060	1.24
Booth-4	8	ASAP7	21.2	468	1.75	15.8	9.04	63.2
Booth-4	16	sky130	5,921	7,339	0.13	2,707	21,589	0.369
Booth-4	16	ASAP7	82.7	845	1.05	55.5	52.6	18.0
Tool a×b	4	sky130	450	1,689	0.43	145	338	6.89
Tool a×b	4	ASAP7	5.6	251	2.82	3.57	1.26	280
Tool a×b	8	sky130	2,380	3,722	0.23	526	2,294	1.90
Tool a×b	8	ASAP7	31.3	495	1.67	18.7	11.1	53.6
Tool a×b	16	sky130	9,519	7,402	0.12	2,603	20,927	0.384
Tool a×b	16	ASAP7	132	953	0.95	74.0	78.1	13.5
Dadda + KS	16	sky130	7,802	5,351	0.17	2,738	16,394	0.365
Dadda + KS	16	ASAP7	109	676	1.28	71.1	55.4	14.1
Booth-4 + KS	16	sky130	6,898	4,774	0.18	2,242	12,133	0.446
Booth-4 + KS	16	ASAP7	86.8	595	1.43	51.6	36.0	19.4
Array, 4-stage	16	sky130	12,138	5,573	0.16	17,876	111,016	0.056
Array, 4-stage	16	ASAP7	180	343	2.24	292	130	3.43

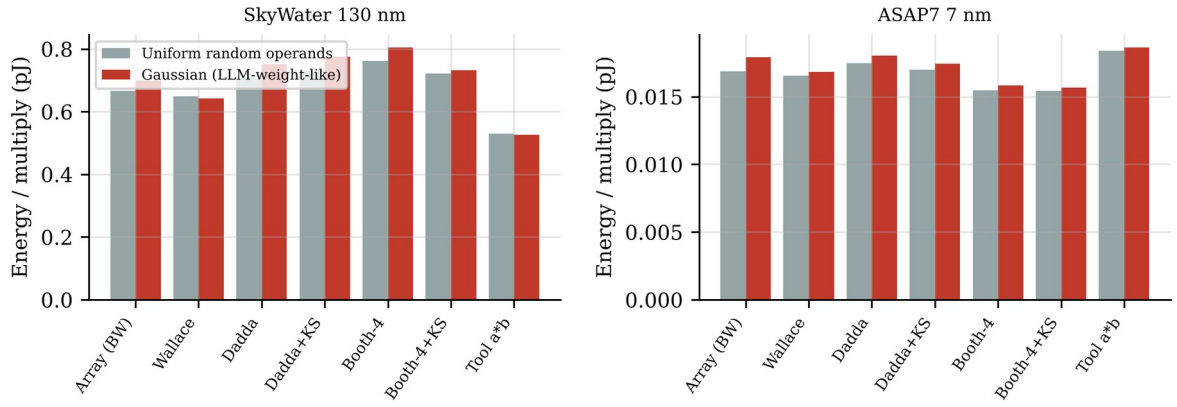


Fig. 11. Energy per multiply for uniformly random and Gaussian (LLM-weight-like) operands.

Operand statistics (Fig. 11). LLM weights are concentrated near zero. In two's complement, small negative values have long runs of leading ones, so successive operands of opposite sign toggle the whole upper part of the partial-product array: with Gaussian operands the array multiplier consumes 1.06× the energy it consumes for uniform operands in ASAP7 (1.05× in sky130), whereas Booth recoding — whose recoder produces zero partial products for runs of identical bits — changes by 1.02× (ASAP7). Data-aware choices such as sign-magnitude weight encoding or Booth recoding therefore deserve attention in an engine dedicated to LLM weights.

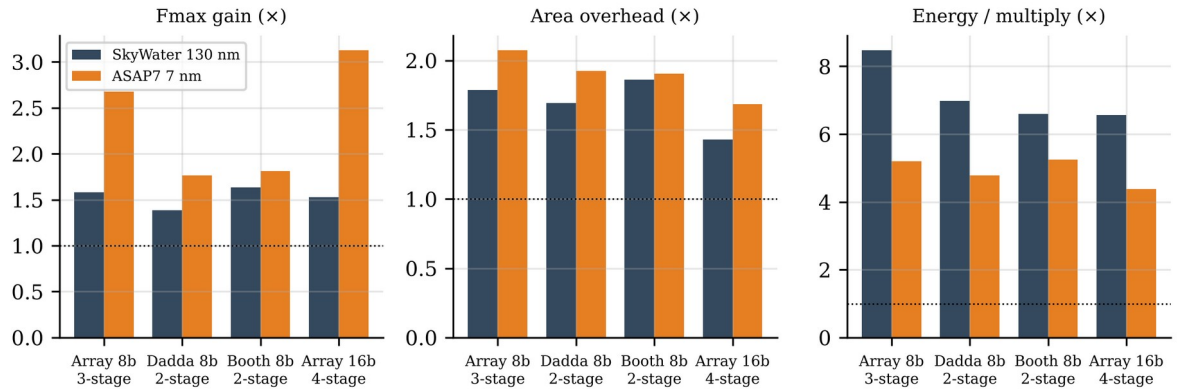


Fig. 12. Effect of pipelining: clock-frequency gain, area overhead and energy per multiply relative to the combinational version.

Pipelining (Fig. 12). Three pipeline cuts raise the ASAP7 array multiplier's clock rate by 2.68× at 107% more area and 419% more energy per multiply; the registers add clock and data switching energy that outweighs the glitch reduction. (Combinational designs are reported without input/output registers, whereas every pipeline register — including operand copies — is counted in the pipelined designs; liberty data put the clock-pin energy alone at ≈20 fJ per edge for a sky130 flip-flop.) Since a near-flash engine is sized by bandwidth rather than by clock rate (Section 7.6), pipelining is worthwhile only if it reduces the number of MAC units — and hence area — more than it adds per unit.

Table 11. MAC engines for the base die (area-driven synthesis, Gaussian operands; the systolic array performs 32 MACs per cycle and its E/op is per MAC).

Engine	Library	Area (µm²)	Cells	T_clk (ps)	f_max (GHz)	E/MAC (fJ)	Leak (µW)	GOPS/ mm²	TOPS/ W
Unsigned 4x4 array (FP8 mantissa)	sky130	353	48	2,564	0.39	132	0.000	1,105	7.59
Unsigned 4x4 array (FP8 mantissa)	ASAP7	4.9	56	328	3.05	3.95	0.002	616,708	253
INT8 MAC PE (array mult + 32b acc)	sky130	4,086	475	7,988	0.13	4,108	0.002	31	0.243
INT8 MAC PE (array mult + 32b acc)	ASAP7	61.2	544	713	1.40	64.3	0.028	22,931	15.5
FP8 E4M3 MAC (exact 48b acc)	sky130	6,071	754	15,537	0.06	6,142	0.002	11	0.163
FP8 E4M3 MAC (exact 48b acc)	ASAP7	89.3	841	1,208	0.83	87.9	0.037	9,275	11.4
4x8 INT8 systolic array	sky130	128,223	15073	7,993	0.13	3,042	0.049	31	0.329
4x8 INT8 systolic array	ASAP7	1,911	17232	1,233	0.81	49.3	0.846	13,583	20.3

MAC engines (Table 11). The FP8 E4M3 MAC with exact 48-bit accumulation costs 1.46× the area and 1.37× the energy of the INT8 PE in ASAP7 (1.49× and 1.49× in sky130), although its significand multiplier is only 4×4: the alignment shifter and wide accumulator dominate. Its critical path is the stage that combines the barrel shifter with conditional negation (15.5 ns in sky130), which is the obvious candidate for an extra pipeline cut. The 4×8 systolic array delivers 13,583 GOPS/mm² and 20.3 TOPS/W in ASAP7.

7.6 System-level comparison: conventional HBM, HBF with far compute, and near-flash compute

We now place the engines in the three systems of Fig. 1 and Fig. 7: **S1**, a conventional package with weights in HBM and MACs on the accelerator die (no HBF); **S2**, HBF with weights streamed over the interposer to MACs on the accelerator; and **S3**, HBF with MACs in the base die. Table 12 lists the energy parameters. MAC energy, area and clock rate come from Tables 8–11 (accelerator MACs are taken from ASAP7; base-die MACs from either library, bracketing the unknown base-die process).

Table 12. System-model parameters (defaults; all are swept in Figs. 14–16).

Parameter	Default	Range explored	Meaning
e_HBM	4 pJ/bit	—	HBM access + I/O to accelerator (HBM2-class [25])
e_NAND	2 pJ/bit	0.5–10	NAND sense + page-buffer read (assumption)
e_TSV	0.1 pJ/bit	—	Core die → base die
e_PHY	0.5 pJ/bit	0.25–3	Base die → accelerator over interposer
e_SRAM	0.2 pJ/bit	—	Operand staging on the accelerator
e_buf	0.05 pJ/bit	—	Page buffer → near-flash MAC
β_stack	0.8 TB/s	—	External HBF read bandwidth per stack
γ	1	1–4	Internal-array / external bandwidth ratio
P_budget	5 W	1–15	Compute power allowed in one base die
A_budget	20 mm²	—	Base-die area available for compute

For a decode step with batch B, the weight-GEMV energy per generated token is

$$E_{token} = [\, 8\, W_{touched}(B) \cdot e_{move}^{(S)} + B \cdot P_{act} \cdot E_{MAC} + E_{act}^{(S)} \,] \,/\, B \tag{12a}$$

$$e^{(S1)} = e_{HBM} + e_{SRAM}, \quad e^{(S2)} = e_{NAND} + e_{TSV} + e_{PHY} + e_{SRAM}, \quad e^{(S3)} = e_{NAND} + e_{TSV} + e_{buf} \tag{12b}$$

where $E_{act}^{(S3)}$ accounts for activations and FP32 partial sums that must still cross the interposer in S3. A base-die engine must sustain a MAC rate of $\gamma\beta_{stack} \cdot B$ (every weight byte is used once per sequence in the batch), which requires $U = \gamma\beta_{stack} B / f_{clk}$ MAC units; the largest batch it can serve within the budgets is therefore

$$B_{max} = min(\, P_{budget} / (\gamma\, \beta_{stack}\, E_{MAC}), \quad A_{budget} \cdot f_{clk} / (\gamma\, \beta_{stack}\, A_{MAC}) \,) \tag{13}$$

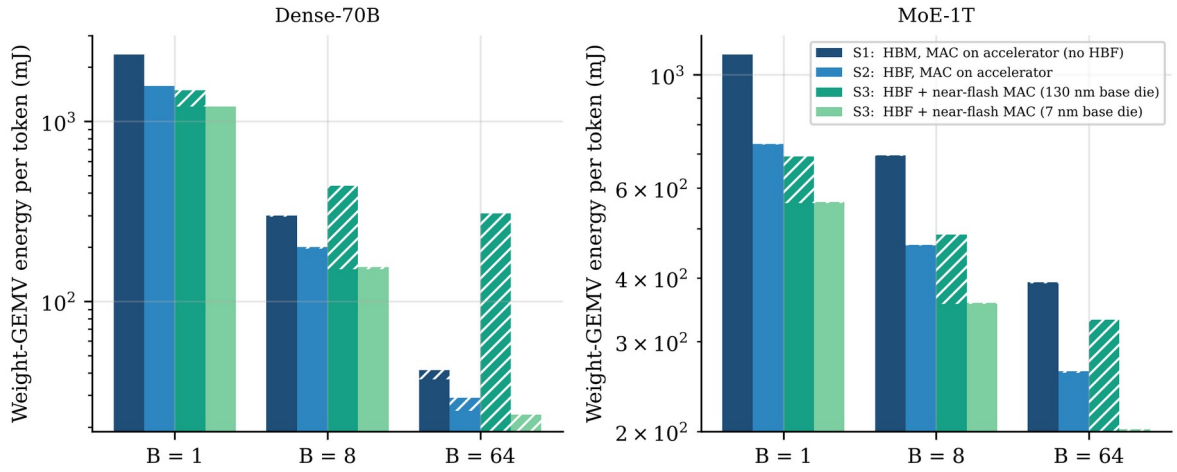


Fig. 13. Weight-GEMV energy per generated token for S1 (HBM, no HBF), S2 (HBF, far compute) and S3 (HBF, near-flash MACs in a 130 nm or 7 nm base die); solid = data movement, hatched = MAC computation.

Energy (Fig. 13, Table 13). As in general-purpose computing [59], data movement dominates. For MoE-1T at $B = 1$, moving the touched weights costs 1,095 mJ per token in S1 and 729.9 mJ in S2, while the MACs themselves cost only 2.06 mJ — a ratio of about 532:1. S3 removes the interposer and accelerator-staging energy from the weight path and reduces the per-token energy to 562.6 mJ with a 7 nm base die (23% below S2 and 49% below the conventional HBM system) and to 692.0 mJ with a 130 nm base die. As the batch grows, weight traffic is amortised and MAC energy becomes visible: at $B = 64$ a 130 nm base-die engine costs 131.46 mJ per token in computation and the total is 331.31 mJ versus 262.18 mJ for S2 and 392.24 mJ for S1, so **the multiplier's energy efficiency — and the base-die process — decides whether near-flash compute remains advantageous at large batch**.

Table 13. Weight-GEMV energy per generated token (mJ) by system and batch.

Model	B	System	Movement	MAC	Act./psum	Total
Dense-70B	1	S1: HBM, MAC on accelerator (no HBF)	2,352	4.503	0.0000	2,357
Dense-70B	1	S2: HBF, MAC on accelerator	1,568	4.503	0.0000	1,573
Dense-70B	1	S3: HBF + near-flash MAC (130 nm base die)	1,204	287.579	0.1376	1,492
Dense-70B	1	S3: HBF + near-flash MAC (7 nm base die)	1,204	4.503	0.1376	1,209
Dense-70B	8	S1: HBM, MAC on accelerator (no HBF)	294.00	4.503	0.0000	298.50
Dense-70B	8	S2: HBF, MAC on accelerator	196.00	4.503	0.0000	200.50
Dense-70B	8	S3: HBF + near-flash MAC (130 nm base die)	150.50	287.579	0.1376	438.22
Dense-70B	8	S3: HBF + near-flash MAC (7 nm base die)	150.50	4.503	0.1376	155.14
Dense-70B	64	S1: HBM, MAC on accelerator (no HBF)	36.75	4.503	0.0000	41.25
Dense-70B	64	S2: HBF, MAC on accelerator	24.50	4.503	0.0000	29.00
Dense-70B	64	S3: HBF + near-flash MAC (130 nm base die)	18.81	287.579	0.1376	306.53
Dense-70B	64	S3: HBF + near-flash MAC (7 nm base die)	18.81	4.503	0.1376	23.45
MoE-1T	1	S1: HBM, MAC on accelerator (no HBF)	1,095	2.059	0.0000	1,097
MoE-1T	1	S2: HBF, MAC on accelerator	729.87	2.059	0.0000	731.93
MoE-1T	1	S3: HBF + near-flash MAC (130 nm base die)	560.43	131.465	0.1049	692.00
MoE-1T	1	S3: HBF + near-flash MAC (7 nm base die)	560.43	2.059	0.1049	562.60
MoE-1T	8	S1: HBM, MAC on accelerator (no HBF)	693.62	2.059	0.0000	695.68
MoE-1T	8	S2: HBF, MAC on accelerator	462.41	2.059	0.0000	464.47
MoE-1T	8	S3: HBF + near-flash MAC (130 nm base die)	355.07	131.465	0.1049	486.64
MoE-1T	8	S3: HBF + near-flash MAC (7 nm base die)	355.07	2.059	0.1049	357.23
MoE-1T	64	S1: HBM, MAC on accelerator (no HBF)	390.18	2.059	0.0000	392.24
MoE-1T	64	S2: HBF, MAC on accelerator	260.12	2.059	0.0000	262.18
MoE-1T	64	S3: HBF + near-flash MAC (130 nm base die)	199.74	131.465	0.1049	331.31
MoE-1T	64	S3: HBF + near-flash MAC (7 nm base die)	199.74	2.059	0.1049	201.90

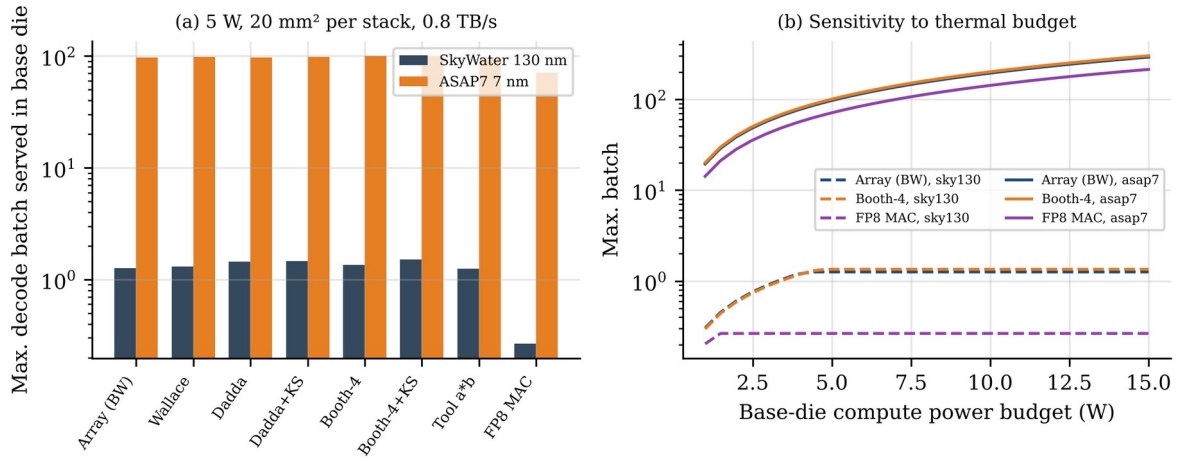


Fig. 14. (a) Largest decode batch a base-die engine can serve per stack (Equation (13)) for each multiplier architecture and library at 5 W and 20 mm²; (b) sensitivity of B_max to the thermal budget.

How many MACs fit under the NAND? (Fig. 14, Table 14). At 0.8 TB/s per stack every unit of batch requires 0.051 W and 0.034 mm² of INT8 array-multiplier PEs in ASAP7, but 3.29 W and 15.87 mm² in sky130. Under a 5 W, 20 mm² budget, a 7 nm base die can therefore serve decode batches up to $B_{max} \approx 97$ (limited by power), whereas a 130 nm base die is limited to $B_{max} \approx 1.3$ (limited by area). The best architecture for this purpose is the Booth-4+KS in ASAP7 ($B_{max} \approx 101$) and the Booth-4+KS in sky130 ($B_{max} \approx 1.5$). The FP8 MAC reduces B_{max} to 71 (ASAP7) and 0.3 (sky130). **The base-die process node, not the multiplier topology, is the first-order design decision**: it changes B_{max} by more than an order of magnitude, while the choice among multiplier architectures changes it by only 20% (sky130) and 5% (ASAP7), because the accumulator, pipeline and pass-through registers of a PE consume most of its energy.

Table 14. Near-flash MAC capacity per HBF stack (0.8 TB/s, γ = 1, 5 W, 20 mm²). PE = multiplier + shared accumulator/registers.

PE multiplier	Library	PE area (μm²)	T_clk (ps)	E/MAC (fJ)	W per unit B	mm² per unit B	B_max (power)	B_max (area)	B_max
Array (BW)	sky130	4,086	4,856	4,108	3.287	15.874	1.5	1.3	1.3
Wallace	sky130	4,210	4,534	4,052	3.241	15.273	1.5	1.3	1.3
Dadda	sky130	4,055	4,265	4,161	3.329	13.835	1.5	1.4	1.4
Dadda+KS	sky130	4,049	4,218	4,184	3.347	13.661	1.5	1.5	1.5
Booth-4	sky130	3,682	5,040	4,215	3.372	14.847	1.5	1.3	1.3
Booth-4+KS	sky130	3,836	3,680	4,141	3.313	11.294	1.5	1.8	1.5
Tool a×b	sky130	4,569	4,359	3,935	3.148	15.934	1.6	1.3	1.3
FP8 MAC	sky130	6,071	15,537	6,142	4.913	75.460	1.0	0.3	0.3
Array (BW)	ASAP7	61.2	695	64.3	0.051	0.034	97.2	587.6	97.2
Wallace	ASAP7	64.6	592	63.2	0.051	0.031	98.8	653.6	98.8
Dadda	ASAP7	60.7	549	64.5	0.052	0.027	97.0	749.2	97.0
Dadda+KS	ASAP7	60.9	578	63.9	0.051	0.028	97.9	710.8	97.9
Booth-4	ASAP7	58.0	571	62.2	0.050	0.026	100.4	754.7	100.4
Booth-4+KS	ASAP7	58.3	537	62.1	0.050	0.025	100.7	797.4	100.7
Tool a×b	ASAP7	68.0	598	65.1	0.052	0.033	96.1	615.1	96.1
FP8 MAC	ASAP7	89.3	1,208	87.9	0.070	0.086	71.1	231.9	71.1

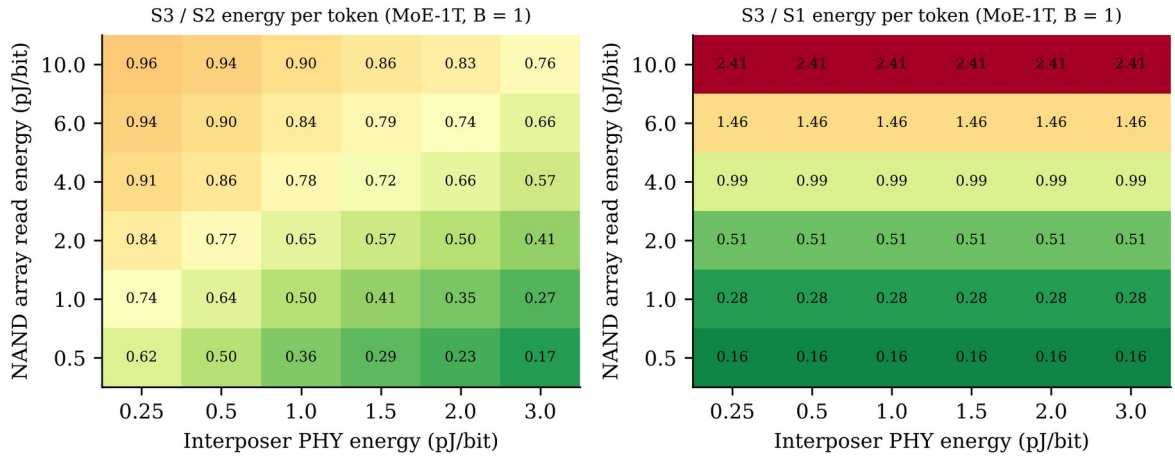


Fig. 15. Sensitivity of near-flash compute (S3) energy per token relative to S2 (left) and to the conventional HBM system S1 (right), over NAND array read energy and interposer PHY energy (MoE-1T, B = 1, 7 nm base die). Values below 1 favour near-flash compute.

Sensitivity (Fig. 15). Relative to S2, near-flash compute is always beneficial — the ratio ranges from 0.17 to 0.96 over the explored grid — because it removes PHY and staging energy without adding any. Relative to the conventional HBM system the answer depends on the NAND read energy: at the default 0.5 pJ/bit PHY, S3 is better than S1 whenever $e_{\mathrm{NAND}} \leq 4.0$ pJ/bit, and the ratio spans 0.16–2.41 over the grid. The energy-per-bit of the NAND read path is thus the single most important unknown for HBF as an energy-efficient — not merely capacity-efficient — technology.

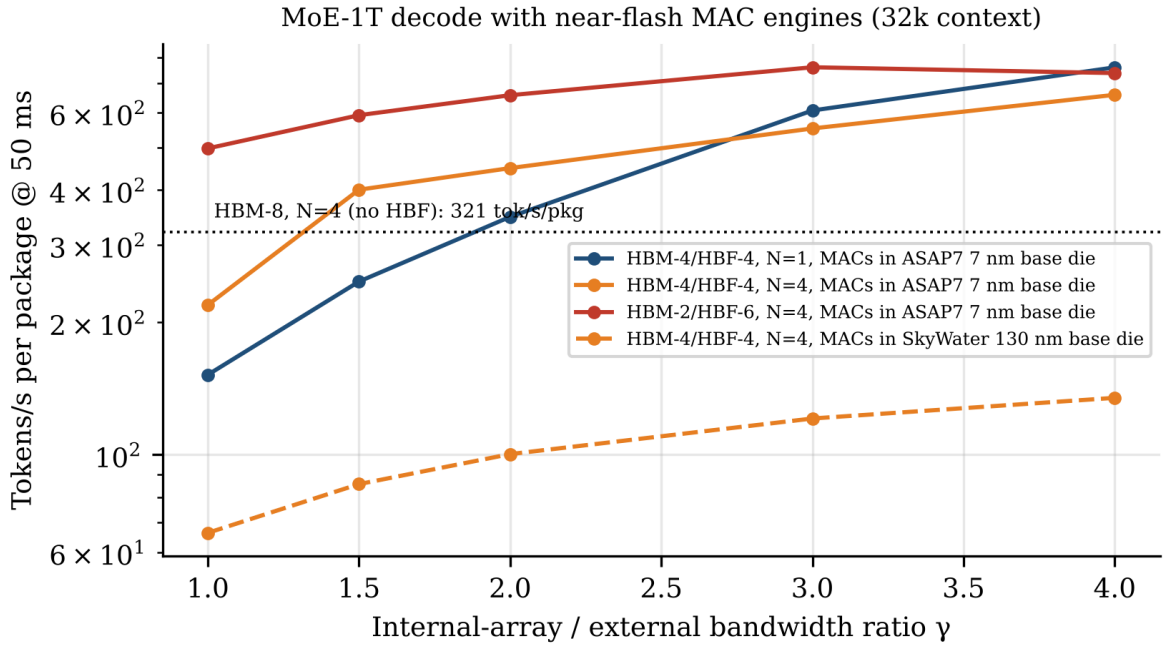


Fig. 16. MoE-1T decode throughput per package (TPOT ≤ 50 ms, 32k context) when near-flash MAC engines consume weights at γ times the external HBF bandwidth, limited to the batch B_max they can serve. Dotted line: conventional HBM-8 system on four packages.

Throughput (Fig. 16). If the base die can read the arrays faster than its external interface allows (γ > 1) — plausible because internal TSV bandwidth is not limited by interposer beachfront — near-flash engines convert that internal bandwidth directly into decode throughput. For MoE-1T on four HBM-4/HBF-4 packages, throughput rises from 219 tokens/s per package at γ = 1 to 659 at γ = 4 with 7 nm engines, compared with 321 for the conventional HBM-8 system on the same four packages; a single

package reaches 761 tokens/s at γ = 4 (from 152). With 130 nm engines the batch limit of Equation (13) caps the gain at 135 tokens/s per package even at γ = 4. The curves are not always monotonic in γ: each unit of batch costs γ times more MAC power (Equation (13)), so a faster array read shrinks the batch the base die can serve. Because the accelerator's own MACs remain available, a real system would fall back to far compute (S2) for batches above B_{max}; the near-flash path should be read as an additional, energy-efficient mode for small batches rather than a replacement. Near-flash multipliers are therefore an *enabler* of the HBF capacity advantage only when the base die is fabricated in a node advanced enough to host thousands of MAC units within a few watts.

7.7 Design guidelines

G1. **Choose the base-die node first.** It changes the sustainable near-flash batch by more than an order of magnitude (Table 14); multiplier topology changes it by at most about 20%.

G2. **Use array or Booth multipliers at INT8/FP8 widths.** At ≤ 8 bits the regular array multiplier is within 33% of the best design on area, clock period and energy and is the easiest to place between TSV fields; Booth recoding (with a parallel-prefix CPA) is the best single choice in ASAP7 and becomes clearly superior at 16 bits.

G3. **Optimise the accumulator and registers, not only the multiplier.** In a PE they dominate energy; narrower accumulators, clock gating and fewer pass-through registers yield more than a better multiplier.

G4. **Add a parallel-prefix CPA where it shrinks the engine.** Kogge–Stone shortens the critical path; in a bandwidth-sized engine a faster clock means fewer MAC units, so the CPA pays off only if the area it saves per unit of batch exceeds its own cost.

G5. **Pipeline FP8 alignment, not the multiplier.** In the FP8 MAC the shifter-plus-negation stage, not the 4×4 significand multiplier, limits the clock.

G6. **Keep only GEMV in the base die.** Attention over the KV cache stays in HBM next to the accelerator (Table 1); the near-flash engine serves dense and expert FFN weights, where the weight-to-activation ratio is largest.

8. The System Architecture Debate

The inference memory hierarchy is the subject of an active debate between at least five architectural positions. Table 6 summarises them; we then argue where each is strongest, using the results of Section 6.

Table 15. Competing architectural positions for the LLM inference memory hierarchy.

Position	Core idea	Strengths	Weaknesses	Best regime
A. HBM scale-out	More HBM per package (HBM4/4E), shard across many packages	Highest bandwidth; mature software	Capacity cost; packages sized by memory, not compute	Latency-critical serving of models that fit
B. On-package HBF	Flash stacks beside HBM on interposer	8–16× capacity/stack; fewer packages	Read-only practicality; tR; endurance; immature	Large MoE, small N, prefill nodes
C. CXL / host pools	Off-package DRAM or flash over CXL/PCIe	Elastic, shareable capacity	Link bandwidth ≪ HBM	KV spill, prefix caches, batch jobs
D. Near/in-memory compute	Compute inside HBM or flash base dies	Avoids moving weights; high internal BW	Programmability; thermal; vendor lock-in	Attention, GEMV, expert FFN offload

Position	Core idea	Strengths	Weaknesses	Best regime
E. SRAM-centric dataflow	Distribute weights across many chips' SRAM	Lowest latency per token	Huge chip count for TB-scale models	Low-latency serving of mid-size models

Position A versus B. Our results support neither extreme. For models that fit in HBM, Position A is strictly better (Section 6.2, Dense-70B). For trillion-parameter MoE models at small-to-moderate package counts — the regime of on-premises, sovereign, enterprise and edge-datacentre deployments, where package count is fixed by budget or power — Position B is the only option at $N \leq 2$ and leads by 1.75–4.2× at $N = 4$ for long contexts. At hyperscale, where operators can deploy as many packages as needed, Position A remains competitive on throughput per package, and the argument for HBF shifts to cost per token, which depends on HBF pricing not yet public.

Position C. CXL-attached memory [33], [34] offers capacity elasticity and sharing across packages but at link bandwidths one to two orders of magnitude below HBM. Our decode analysis implies it cannot hold per-step weight traffic for interactive serving; it is, however, an excellent home for warm prefix caches and preempted KV, and is complementary to HBF.

Position D. Processing-near-memory [35], [36] and in-flash computation [31], [42] would invert the trade-off identified in Section 6.2: if the expert FFN computation for a token were performed in the HBF base die, only activations — kilobytes rather than gigabytes — would cross the interface, removing HBF bandwidth as the binding constraint. The base die that HBF already requires is a natural host for such logic; Section 7 quantifies this option with synthesised multiplier engines and shows that its viability hinges on the base-die process node and on the NAND read energy, while thermal limits next to NAND and the need for a programmable, standardised interface remain open problems.

Position E. SRAM-centric designs [43] minimise TPOT but require a number of chips proportional to model size; for trillion-parameter models the chip count, and thus capital cost, is prohibitive. They and HBF sit at opposite corners of the capacity–latency plane.

8.1 A heterogeneous fleet: HBF for prefill, HBM for decode

Combining Sections 6.2 and 6.4 yields a concrete architectural proposal. Prefill is compute-bound when chunks are large, needs little *persistent* KV capacity (KV is shipped to decode nodes immediately [9]–[11]), and tolerates HBF weight bandwidth once $L \geq L^{*}$. Decode is bandwidth- and KV-capacity-bound and is hurt by moving weight traffic to flash unless capacity, not bandwidth, is the binding constraint. We therefore propose that **prefill pools be built from HBF-heavy packages** (e.g., HBM-2/HBF-6), which can hold an entire trillion-parameter model per package and thus prefill without any model-parallel communication, while **decode pools use HBM-rich packages**, or HBM-4/HBF-4 hybrids when the deployment size is fixed and small. This assignment uses each memory technology where its weaknesses are masked by the phase's characteristics, and it is compatible with KV-centric serving architectures already in production [11].

9. Open Research Challenges

- **Array-level parallelism (VLSI).** Achieving $\approx 10^3$ concurrent reads per stack (Section 6.6) at acceptable area, power and peripheral-circuit overhead; low-latency sensing schemes and sub-array partitioning in 3D NAND.
- **Thermal co-design.** Retention, bit-error rate and ECC strength of NAND operated next to a kilowatt-class accelerator; interposer floor-planning and thermal isolation.

- **Interface standardisation.** An HBM-compatible PHY with a read-optimised command set (bulk descriptors, sealed regions) to allow mixed HBM/HBF packages from multiple vendors [30].
- **Expert-aware scheduling.** Batch formation that minimises p_{touch} (Equation (5)) through routing-affinity grouping; hot-expert caching policies between HBM and HBF.
- **Speculative expert prefetch.** Predicting next-layer expert selection to hide t_R in MoE decode without wasting flash bandwidth.
- **Reliability under read-heavy stress.** Read-disturb characterisation at billions of reads per block, and the background relocation write budget it implies.
- **Compute in the base die.** Moving from the fixed-function engines of Section 7 to programmable near-flash GEMV/FFN engines, with physical design (place-and-route, IR drop, thermal maps) of MAC tiles between TSV fields.
- **Validation.** Cycle-accurate and silicon-level validation of the analytical model presented here, including energy per token, which this study does not quantify.

10. Conclusion

High Bandwidth Flash reframes the LLM inference memory problem from "how much HBM can be afforded" to "which data deserves DRAM". By deriving and evaluating the governing equations for prefill and decode on a tiered HBM–HBF package, this paper has shown that HBF is not a drop-in substitute for HBM: it lowers per-package decode throughput for models that already fit in HBM, and its value collapses if its read bandwidth falls well below HBM's or if it is used for mutable data. Where it is used for what flash does well — holding enormous, read-mostly weight sets — it changes what is feasible: a trillion-parameter MoE fits in a single package, capacity-constrained deployments gain 1.75–4.2× throughput per package, prefill nodes can be built without model parallelism, and daily model updates consume only a small fraction of endurance. The decisive technical questions are now at the device and circuit level — sub-array parallelism, latency and thermal robustness — and at the system level — expert-aware scheduling and prefill–decode disaggregation. At circuit level, the synthesised multiplier engines show that integrating array or Booth multipliers into the HBF base die is technically straightforward and cuts weight-GEMV energy by eliminating interposer traffic, but that the batch such engines can serve is set by the base-die process node and thermal budget far more than by multiplier topology. We hope the analytical framework and open model presented here help both communities reason about them quantitatively.

Declarations

Funding. This research received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Conflict of interest. The author declares no competing interests.

Data and code availability. The analytical models (hbf_model.py, hw/nmc_model.py), the multiplier generator (hw/gen_mult.py), all Verilog RTL and testbenches, the synthesis and power flow (hw/syn_flow.py) and all raw results (results.json, hw/results/) are available from the author's repository alongside this manuscript.

Author contribution. M. Nisha Angeline: conceptualisation, methodology, analysis, visualisation, writing — original draft, review and editing.

References

[1] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.

[2] J. Kaplan *et al.*, "Scaling laws for neural language models," arXiv:2001.08361, 2020.

[3] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, 2024.

[4] R. Pope *et al.*, "Efficiently scaling transformer inference," in *Proc. Mach. Learn. Syst. (MLSys)*, 2023.

[5] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[6] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM Symp. Operating Syst. Principles (SOSP)*, 2023.

[7] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *Proc. USENIX OSDI*, 2022.

[8] A. Agrawal *et al.*, "Taming throughput–latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. USENIX OSDI*, 2024.

[9] Y. Zhong *et al.*, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. USENIX OSDI*, 2024.

[10] P. Patel *et al.*, "Splitwise: Efficient generative LLM inference using phase splitting," in *Proc. ACM/IEEE Int. Symp. Comput. Archit. (ISCA)*, 2024.

[11] R. Qin *et al.*, "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," in *Proc. USENIX Conf. File Storage Technol. (FAST)*, 2025.

[12] Y. Sheng *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023.

[13] K. Alizadeh *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[14] R. Y. Aminabadi *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, 2022.

[15] N. Shazeer *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[16] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.

[17] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," arXiv:2405.04434, 2024.

[18] DeepSeek-AI, "DeepSeek-V3 technical report," arXiv:2412.19437, 2024.

[19] J. Ainslie *et al.*, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023.

[20] N. Shazeer, "Fast transformer decoding: One write-head is all you need," arXiv:1911.02150, 2019.

[21] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. NeurIPS*, 2022.

[22] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. ICML*, 2023.

[23] JEDEC Solid State Technology Association, *High Bandwidth Memory DRAM (HBM3)*, JESD238, 2022.

[24] D. U. Lee *et al.*, "A 1.2 V 8 Gb 8-channel 128 GB/s high-bandwidth memory (HBM) stacked DRAM with effective microbump I/O test methods using 29 nm process and TSV," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, 2014, pp. 432–433.

[25] M. O'Connor *et al.*, "Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2017.

[26] R. Micheloni, L. Crippa, and A. Marelli, *Inside NAND Flash Memories*. Dordrecht, The Netherlands: Springer, 2010.

[27] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Error characterization, mitigation, and recovery in flash-memory-based solid-state drives," *Proc. IEEE*, vol. 105, no. 9, pp. 1666–1704, 2017.

[28] H. Tanaka *et al.*, "Bit cost scalable technology with punch and plug process for ultra high density flash memory," in *Proc. IEEE Symp. VLSI Technol.*, 2007, pp. 14–15.

[29] Sandisk Corporation, "Sandisk unveils High Bandwidth Flash (HBF) memory technology for AI inference," Investor Day press release, Feb. 2025.

[30] Sandisk Corporation and SK hynix, "Memorandum of understanding to standardize High Bandwidth Flash (HBF)," press release, Aug. 2025.

[31] Z. Yu *et al.*, "Cambricon-LLM: A chiplet-based hybrid architecture for on-device inference of 70B LLM," in *Proc. IEEE/ACM MICRO*, 2024.

[32] X. Pan *et al.*, "InstInfer: In-storage attention offloading for cost-effective long-context LLM inference," arXiv preprint, 2024.

[33] H. Li *et al.*, "Pond: CXL-based memory pooling systems for cloud platforms," in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2023.

[34] D. Das Sharma, R. Blankenship, and D. S. Berger, "An introduction to the Compute Express Link (CXL) interconnect," *ACM Comput. Surv.*, 2024.

[35] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "Processing data where it makes sense: Enabling in-memory computation," *Microprocess. Microsyst.*, vol. 67, pp. 28–41, 2019.

[36] S. Lee *et al.*, "Hardware architecture and software stack for PIM based on commercial DRAM technology," in *Proc. ACM/IEEE ISCA*, 2021.

[37] N. P. Jouppi *et al.*, "TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proc. ACM/IEEE ISCA*, 2023.

[38] J. D. C. Little, "A proof for the queuing formula: L = λW," *Oper. Res.*, vol. 9, no. 3, pp. 383–387, 1961.

[39] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *Proc. ICLR*, 2023.

[40] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. MLSys*, 2024.

[41] J. Hoffmann *et al.*, "Training compute-optimal large language models," in *Proc. NeurIPS*, 2022.

[42] J. Park *et al.*, "Flash-Cosmos: In-flash bulk bitwise operations using inherent computation capability of NAND flash memory," in *Proc. IEEE/ACM MICRO*, 2022.

[43] D. Abts *et al.*, "Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads," in *Proc. ACM/IEEE ISCA*, 2020.

[44] Kimi Team, "Kimi K2: Open agentic intelligence," arXiv preprint, 2025.

[45] C. R. Baugh and B. A. Wooley, "A two's complement parallel array multiplication algorithm," *IEEE Trans. Comput.*, vol. C-22, no. 12, pp. 1045–1047, 1973.

[46] C. S. Wallace, "A suggestion for a fast multiplier," *IEEE Trans. Electron. Comput.*, vol. EC-13, no. 1, pp. 14–17, 1964.

[47] L. Dadda, "Some schemes for parallel multipliers," *Alta Frequenza*, vol. 34, pp. 349–356, 1965.

[48] A. D. Booth, "A signed binary multiplication technique," *Q. J. Mech. Appl. Math.*, vol. 4, no. 2, pp. 236–240, 1951.

[49] O. L. MacSorley, "High-speed arithmetic in binary computers," *Proc. IRE*, vol. 49, no. 1, pp. 67–91, 1961.

[50] P. M. Kogge and H. S. Stone, "A parallel algorithm for the efficient solution of a general class of recurrence equations," *IEEE Trans. Comput.*, vol. C-22, no. 8, pp. 786–793, 1973.

[51] R. P. Brent and H. T. Kung, "A regular layout for parallel adders," *IEEE Trans. Comput.*, vol. C-31, no. 3, pp. 260–264, 1982.

[52] C. Wolf, "Yosys Open SYnthesis Suite," [Online]. Available: https://yosyshq.net/yosys/

[53] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Proc. Int. Conf. Computer Aided Verification (CAV)*, 2010, pp. 24–40.

[54] L. T. Clark *et al.*, "ASAP7: A 7-nm finFET predictive process design kit," *Microelectron. J.*, vol. 53, pp. 105–115, 2016.

[55] SkyWater Technology and Google, "SkyWater SKY130 open-source process design kit," 2020. [Online]. Available: https://github.com/google/skywater-pdk

[56] S. Williams, "Icarus Verilog," [Online]. Available: https://steveicarus.github.io/iverilog/

[57] P. Micikevicius *et al.*, "FP8 formats for deep learning," arXiv:2209.05433, 2022.

[58] U. Kulisch, *Computer Arithmetic and Validity: Theory, Implementation, and Applications*, 2nd ed. Berlin, Germany: De Gruyter, 2013.

[59] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, 2014, pp. 10–14.

[60] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed. New York, NY, USA: Oxford Univ. Press, 2010.